

ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ АВТОМАТИЗАЦИИ ДЕЯТЕЛЬНОСТИ АВТОВОКЗАЛА

(ФИО студента полностью)

Группа 22ИСП2

_____/ Филимонов А.Ю. (ФИО руководителя)
2026г.

_____/ Филимонов А.Ю. (ФИО консультанта)
2026г.

_____/ Филимонов А.Ю. (ФИО консультанта)
2026г.

_____/ Филимонов А.Ю. (ФИО консультанта)
2026г.

_____/ Клюева Г.А. (ФИО представителя администрации)
2026г.

Оценка _____ (_____) _____
дата защиты 2026г.

Пермь, 2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ	4
1 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ.....	8
1.1 Механика работы ИС	8
1.2 Анализ аналогов проекта.....	13
1.3 Используемые технологии и инструменты	15
1.4 Платформа развёртывания	19
2 ПРАКТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ	22
2.1 Проектирование системы	22
2.2 Разработка базы данных	27
2.3 Написание API.....	31
2.4 Разработка сайта.....	35
2.5 Разработка мобильного приложения.....	42
3 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ.....	48
4 ПРАВИЛА И НОРМЫ ТЕХНИКИ БЕЗОПАСНОСТИ.....	57
ЗАКЛЮЧЕНИЕ	61
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	62
ПРИЛОЖЕНИЕ А	64
ПРИЛОЖЕНИЕ Б	65
ПРИЛОЖЕНИЕ В.....	66

					ВКР 09.02.07.22.4.2.3			
Изм.	Лист	№ докум.	Подпись	Дата				
Разработал	Бурдышев И.О.				Проектирование и разработка информационной системы автоматизации деятельности автовокзала	Лит.	Лист	Листов
Проверил	Филимонов А.Ю.						2	62
						ГБПОУ «ПТПИТ» гр.22ИСП2		
Н. Контроль	Филимонов А.Ю.							
Утвердил	Клюева Г.А.							

ПРИЛОЖЕНИЕ Г 67

ПРИЛОЖЕНИЕ Д 68

					ВКР 09.02.07.22.4.2.3			
<i>Изм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Подпись</i>	<i>Дата</i>	<i>Проектирование и разработка информационной системы автоматизации деятельности автовокзала</i>	<i>Лит.</i>	<i>Лист</i>	<i>Листов</i>
<i>Разработал</i>	<i>Бурдышев И.О.</i>							
<i>Проверил</i>	<i>Филимонов А.Ю.</i>						<i>2</i>	<i>62</i>
						<i>ГБПОУ «ПТПИТ» гр.22ИСП2</i>		
<i>Н. Контроль</i>	<i>Филимонов А.Ю.</i>							
<i>Утвердил</i>	<i>Клюева Г.А.</i>							

ВВЕДЕНИЕ

Проблема. В современных условиях развития транспортной инфраструктуры пассажиры автобусных перевозок сталкиваются с рядом существенных трудностей при планировании своих поездок. Устаревшие системы информирования о расписаниях, отсутствие единой платформы для бронирования и оплаты билетов, а также невозможность получения актуальной информации о задержках рейсов в реальном времени создают значительные неудобства для пользователей. Традиционные методы покупки билетов через кассы автовокзалов требуют физического присутствия и зачастую связаны с длительным ожиданием в очередях. Кроме того, отсутствие мобильной альтернативы не позволяет пассажирам оперативно управлять своими бронированиями и получать электронные билеты. Со стороны персонала автовокзала также наблюдается проблема неэффективного управления маршрутами и отсутствия оперативной связи между водителями и диспетчерами, что приводит к несвоевременному информированию пассажиров об изменениях в расписании. Все эти факторы негативно сказываются на качестве обслуживания и снижают привлекательность автобусных перевозок как вида транспорта.

Актуальность. В связи с вышеизложенным, разработка комплексной информационной системы для автовокзала представляет собой крайне актуальную задачу. Создание единой платформы, объединяющей веб-сайт для пассажиров, мобильное приложение и инструменты управления для персонала, позволит существенно улучшить качество обслуживания и оптимизировать рабочие процессы автовокзала. Информационная система должна предоставлять пассажирам возможность удалённого бронирования и оплаты билетов, просмотра актуального расписания маршрутов, получения электронных билетов с QR-кодами, а также сохранения избранных маршрутов в личном кабинете. Для администраторов система должна обеспечивать полное управление маршрутами, расписаниями и учётными записями

					ВКР 09.02.07.22.4.2.3	Лист
						4
Изм.	Лист	№ документа	Подпись	Дата		

сотрудников. Водителям необходима возможность просмотра назначенных маршрутов и оперативной связи с оператором через встроенный чат. Операторам, в свою очередь, требуется инструмент для мониторинга задержек рейсов и оперативной корректировки расписания. Реализация ролевой модели доступа с проверкой прав на стороне сервера обеспечит безопасность данных и корректное функционирование всех компонентов системы.

Для разработки информационной системы выбран современный технологический стек. Фронтенд-часть реализована на базе NextJS с использованием библиотеки компонентов PrimeReact и фреймворка TailwindCSS для стилизации.

Данный выбор обусловлен возможностями серверного рендеринга, обеспечивающего высокую производительность и оптимизацию для поисковых систем, а также богатой экосистемой готовых UI-компонентов, что значительно ускоряет процесс разработки интерфейса.

Бэкенд построен на фреймворке NestJS с применением PrismaORM для работы с базой данных. Предоставляет строгую модульную архитектуру, встроенную поддержку TypeScript и готовые решения для аутентификации, валидации данных и построения REST API, что делает его одним из наиболее подходящих решений для создания масштабируемых серверных приложений.

PrismaORM обеспечивает типобезопасную работу с базой данных PostgreSQL и упрощает миграции схемы данных. Мобильное приложение разработано с использованием .NET MAUI и паттерна MVVM при поддержке CommunityToolkit, что позволяет создать кроссплатформенное приложение с единой кодовой базой для Android и iOS, сохраняя при этом нативную производительность и удобный пользовательский интерфейс.

Объектом квалификационной работы является проектирование и разработка информационной системы автовокзала «АВ-Вокзал».

					<i>ВКР 09.02.07.22.4.2.3</i>	Лист
						5
Изм.	Лист	№ документа	Подпись	Дата		

Предметом квалификационной работы является создание информационной системы автовокзала с использованием фреймворков NextJS, NestJS и платформы .NET MAUI.

Цель работы. разработка и создание информационной системы автовокзала «АВ-Вокзал» на основе современных веб- и мобильных технологий, обеспечивающей автоматизацию процессов бронирования билетов, управления маршрутами и взаимодействия между пассажирами и персоналом.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) проанализировать современные подходы к проектированию и разработке информационных систем для транспортной сферы;
- 2) провести обзор существующих аналогов систем бронирования автобусных билетов и выявить их преимущества и недостатки;
- 3) спроектировать архитектуру информационной системы, включая структуру базы данных, API-интерфейсы и пользовательские интерфейсы;
- 4) разработать и реализовать базу данных PostgreSQL с использованием PrismaORM для хранения информации о маршрутах, рейсах, пользователях, бронированиях и транзакциях;
- 5) разработать REST API на базе фреймворка NestJS, обеспечивающее взаимодействие между фронтендом, мобильным приложением и базой данных;
- 6) создать веб-приложение на базе NextJS с функционалом регистрации и авторизации, бронирования и оплаты билетов, управления избранными маршрутами и личным кабинетом пользователя;
- 7) реализовать административную панель с возможностями управления маршрутами, расписаниями и учётными записями сотрудников;
- 8) разработать мобильное приложение на платформе .NET MAUI, предоставляющее пассажирам возможность оформления билетов, оплаты и предъявления электронных билетов с QR-кодами;

9) реализовать систему ролевого доступа с разделением на роли пользователя, администратора, водителя и оператора с серверной проверкой прав;

10) реализовать систему чата для оперативного взаимодействия между водителями и операторами;

11) реализовать механизм отслеживания задержек рейсов и корректировки расписания в реальном времени.

Теоретическая значимость. В данной работе разработаны требования к архитектуре информационной системы автовокзала, описаны принципы проектирования многоуровневых клиент-серверных приложений с ролевой моделью доступа. Раскрыты возможности применения фреймворков NextJS и NestJS для создания масштабируемых веб-приложений, а также продемонстрированы преимущества использования .NET MAUI для разработки кроссплатформенных мобильных приложений в транспортной сфере.

Практическая значимость. Разработана полнофункциональная информационная система автовокзала «АВ-Вокзал», включающая веб-приложение с системой бронирования и оплаты билетов, мобильное приложение для пассажиров с поддержкой электронных билетов и QR-кодов, административную панель для управления маршрутами и сотрудниками, интерфейсы для водителей с функцией чата, а также рабочие инструменты операторов для мониторинга задержек и корректировки расписания. Система готова к развёртыванию и может быть использована реальными автовокзалами для автоматизации процессов обслуживания пассажиров и управления транспортной инфраструктурой.

1 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ

1.1 Механика работы ИС

Информационная система «АВ-Вокзал» представляет собой комплексное программное решение, предназначенное для автоматизации процессов автовокзала, включая бронирование и оплату билетов, управление маршрутами и расписаниями, взаимодействие между персоналом, а также предоставление пассажирам удобных цифровых сервисов. Архитектура системы построена по клиент-серверному принципу и включает три основных компонента: веб-приложение (фронтенд), серверную часть (бэкенд) и мобильное приложение. Взаимодействие между компонентами осуществляется через REST API [18], что обеспечивает независимость клиентов от серверной реализации и позволяет масштабировать ИС по мере роста нагрузки.

Архитектура информационной системы представлена на рисунке 1.

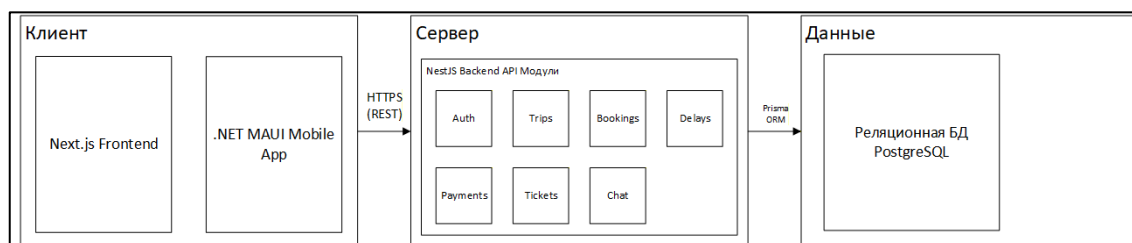


Рисунок 1 — Общая архитектура ИС

Веб-клиент и мобильное приложение выступают в роли потребителей API, бэкенд обрабатывает запросы, выполняет бизнес-логику и взаимодействует с базой данных PostgreSQL [13].

Подобная трёхзвенная архитектура обеспечивает разделение ответственности между слоями, упрощает сопровождение и позволяет вносить изменения в один из компонентов без влияния на остальные.

Веб-приложение разработано как общедоступный интерфейс информационной системы и выполняет функцию основной точки взаимодействия пассажиров с платформой.

Главная страница содержит общую информацию об автовокзале, доступных направлениях и актуальных предложениях.

С главной страницы пользователь переходит на соответствующие функциональные разделы: каталог маршрутов, страницу бронирования, личный кабинет, страницу оплаты.

Процесс бронирования билета представляет собой последовательность шагов, каждый из которых валидируется на стороне сервера.

Пользователь выбирает маршрут из каталога, указывает дату поездки, количество мест и контактные данные.

После формирования брони система резервирует указанные места на выбранном рейсе и создаёт запись о бронировании в базе данных. Механика процесса детализирована на диаграмме последовательности, представленной на рисунке 2.

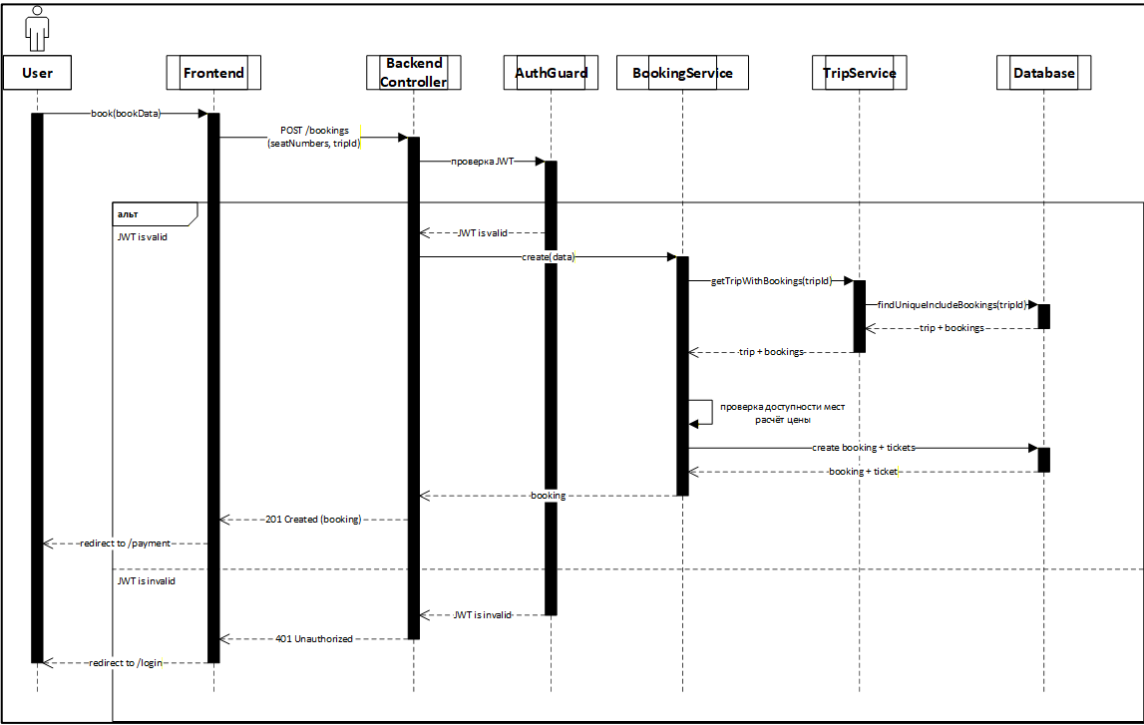


Рисунок 2 — Диаграмма последовательности процесса бронирования

Оплата брони осуществляется через интегрированную платёжную систему.

После успешной оплаты статус бронирования изменяется на «оплачено», и система генерирует электронный билет с уникальным QR-кодом.

QR-код содержит зашифрованную информацию о рейсе, месте, дате и идентификаторе билета, что позволяет проверить его подлинность при посадке в автобус. Электронный билет доступен пользователю как в веб-интерфейсе личного кабинета, так и в мобильном приложении, которое дублирует основной функционал веб-приложения для пассажиров и предоставляет дополнительные возможности, связанные с мобильной спецификой.

Пользователь может оформить билет, оплатить его и предъявить электронный билет или QR-код непосредственно при посадке.

Мобильное приложение обеспечивает автономный доступ к сохранённым билетам, что позволяет пассажиру воспользоваться билетом даже при отсутствии подключения к сети интернет. Архитектура мобильного приложения построена на паттерне MVVM [8], который разделяет пользовательский интерфейс, бизнес-логику и данные, обеспечивая тестируемость и поддерживаемость кода. Диаграмма классов мобильного приложения показана на рисунке 3.

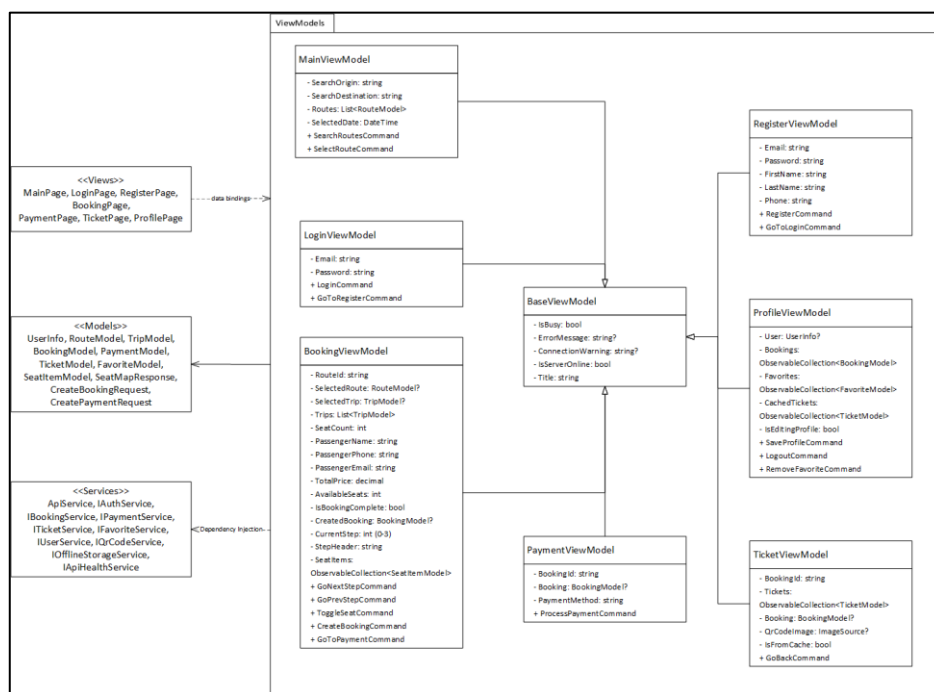


Рисунок 3 — Диаграмма классов мобильного приложения

Система поддерживает ролевую модель доступа с четырьмя основными ролями: пользователь (пассажир), администратор, водитель и оператор.

Каждая роль определяет набор доступных функций и страниц.

Проверка прав доступа выполняется на стороне сервера при каждом запросе к защищённым ресурсам, что исключает возможность обхода ограничений через клиентскую часть.

Администратор обладает полным спектром управленческих функций.

Через административную панель осуществляется добавление, редактирование и удаление маршрутов, включая информацию о начальной и конечной точках, промежуточных остановках, времени отправления и прибытия, стоимости билетов и количестве доступных мест.

Администратор также управляет учётными записями сотрудников: создаёт и редактирует аккаунты других администраторов, водителей и операторов, назначает роли и контролирует доступ. Журнал действий администратора фиксируется в системе для обеспечения аудита.

Функционал водителя включает просмотр назначенных маршрутов, информации о расписании и пассажирах на рейсе.

Ключевой возможностью для водителя является встроенный чат с оператором, позволяющий оперативно сообщать о задержках, изменениях на маршруте, технических проблемах или иных ситуациях, требующих корректировки расписания.

Чат реализован с поддержкой обмена сообщениями в режиме, близком к реальному времени, и сохраняет историю переписки для последующего анализа.

Оператор выполняет функцию диспетчера и отвечает за мониторинг текущего состояния рейсов. Через интерфейс оператора отображается список активных маршрутов, информация о задержках, статусы рейсов и данные о пассажирах. Оператор имеет возможность корректировать время отправления и прибытия, что автоматически обновляет информацию на всех клиентских устройствах. При получении сообщения от водителя о задержке оператор фиксирует её в системе, и обновлённые данные становятся доступны

пассажирам. Механизм обработки задержек и обновления расписания детализирован на диаграмме деятельности, представленной в приложении А.

База данных PostgreSQL выполняет функцию центрального хранилища всей информации системы.

В базе данных хранятся сведения о пользователях, маршрутах, рейсах, бронированиях, транзакциях, сообщениях чата и задержках.

Структура спроектирована с учётом нормализации до третьей нормальной формы, что обеспечивает целостность данных и минимизирует избыточность. Схема базы данных с основными таблицами и связями между ними приведена на рисунке 4.

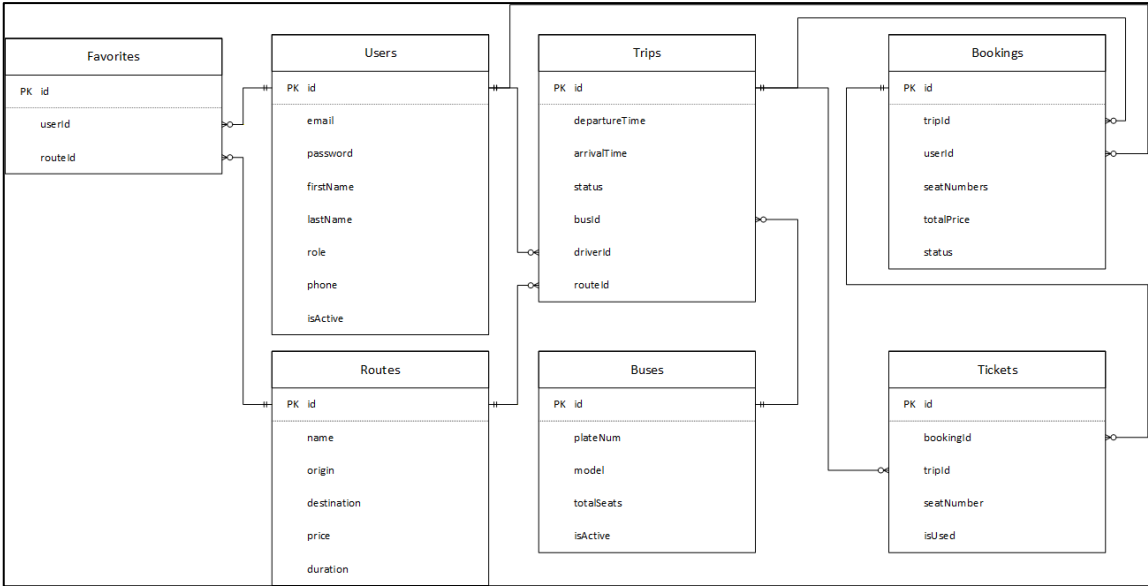


Рисунок 4 — Схема базы данных с основными таблицами

Взаимодействие между всеми компонентами системы координируется через единый API, разработанный на базе фреймворка NestJS [10].

API предоставляет набор конечных точек (endpoints) для аутентификации, управления маршрутами, бронирования, оплаты, чата и мониторинга задержек.

Каждая конечная точка документирована и поддерживает стандартные HTTP-методы. Спецификация основных конечных точек API представлена в приложении Б.

Таким образом, механика работы информационной системы «АВ-Вокзал» охватывает полный цикл взаимодействия пассажиров с платформой

— от поиска маршрута до предъявления электронного билета при посадке, а также обеспечивает эффективное управление автовокзалом со стороны персонала через специализированные интерфейсы и инструменты оперативного взаимодействия.

1.2 Анализ аналогов проекта

На рынке цифровых сервисов для автобусных перевозок представлено значительное количество решений, выполняющих функции бронирования и продажи билетов. Анализ существующих аналогов позволяет выявить преимущества и недостатки каждого подхода, определить оптимальные практики и сформировать требования к разрабатываемой информационной системе. В рамках данного раздела рассмотрены наиболее известные платформы, предоставляющие услуги бронирования автобусных билетов.

Яндекс.Путешествия — агрегатор транспортных услуг, входящий в экосистему сервисов компании Яндекс. Платформа предоставляет пользователям возможность поиска и бронирования билетов на автобусы, поезда и самолёты. Интерфейс агрегатора позволяет фильтровать маршруты по дате, времени, цене и перевозчику. Система интегрирована с множеством автовокзалов и транспортных компаний, что обеспечивает широкий выбор маршрутов. Среди преимуществ платформы следует отметить узнаваемый бренд, развитую инфраструктуру и интеграцию с другими сервисами экосистемы. Однако агрегатор не предоставляет функционала управления автовокзалом, чата между персоналом или системы отслеживания задержек в реальном времени. Платформа ориентирована исключительно на пассажиров и не решает задачи оперативного управления транспортным предприятием.

Tutu.ru — один из крупнейших сервисов бронирования билетов на автобусы и поезда в России. Платформа работает как агрегатор, собирая предложения от различных перевозчиков и предоставляя пользователям единый интерфейс для поиска и покупки билетов. Сервис поддерживает онлайн-оплату, возврат билетов и получение электронных билетов.

					ВКР 09.02.07.22.4.2.3	Лист
Изм.	Лист	№ документа	Подпись	Дата		13

Преимущества Tutu.ru включают широкую базу маршрутов, удобный пользовательский интерфейс и систему уведомлений об изменениях в расписании. К недостаткам относится отсутствие инструментов для управления автовокзалом со стороны администрации, невозможность оперативного взаимодействия между водителями и диспетчерами, а также зависимость от внешних перевозчиков при формировании базы маршрутов.

Busfor — сервис бронирования автобусных билетов, ориентированный на междугородные и международные перевозки. Платформа сотрудничает с транспортными компаниями и предоставляет пользователям возможность поиска маршрутов, сравнения цен и онлайн-бронирования. Сервис поддерживает мобильное приложение, в котором доступны купленные билеты и информация о рейсах. Преимущества Busfor заключаются в специализации на автобусных перевозках и наличии мобильного приложения. Недостатки включают ограниченный набор маршрутов по сравнению с крупными агрегаторами, отсутствие инструментов для управления расписанием со стороны автовокзала и отсутствие системы чата для оперативного взаимодействия персонала.

RedBus — международная платформа бронирования автобусных билетов, работающая в нескольких странах. Сервис предоставляет широкий выбор маршрутов, систему отзывов о перевозчиках, онлайн-оплату и электронные билеты. Мобильное приложение RedBus позволяет управлять бронированиями и получать уведомления о статусе рейса. Среди преимуществ платформы — международное покрытие, система рейтингов перевозчиков и развитый мобильный клиент. К недостаткам относятся отсутствие инструментов управления автовокзалом, невозможность корректировки расписания оператором в реальном времени и отсутствие встроенной системы коммуникации между водителями и диспетчерами.

Анализ рассмотренных аналогов позволяет сделать следующие выводы. Существующие платформы бронирования автобусных билетов ориентированы преимущественно на пассажиров и не решают задачи

					<i>ВКР 09.02.07.22.4.2.3</i>	Лист
						14
Изм.	Лист	№ документа	Подпись	Дата		

комплексного управления автовокзалом. Ни одна из рассмотренных платформ не предоставляет полного набора инструментов для администраторов, водителей и операторов в рамках единой системы. Функционал чата между водителями и операторами, система отслеживания задержек в реальном времени и ролевая модель доступа с серверной проверкой прав отсутствуют в рассмотренных аналогах. Разрабатываемая информационная система «АВ-Вокзал» восполняет данный пробел, объединяя пассажирские сервисы бронирования и оплаты с инструментами оперативного управления автовокзалом в единой платформе.

1.3 Используемые технологии и инструменты

Разработка информационной системы «АВ-Вокзал» потребовала выбора технологического стека, обеспечивающего высокую производительность, масштабируемость, безопасность и удобство сопровождения. Выбор технологий осуществлялся на основе анализа современных решений в области веб-разработки, мобильной разработки и управления базами данных. В данном разделе приведено обоснование выбора каждой технологии, используемой в проекте.

Фронтенд-часть информационной системы реализована с использованием фреймворка NextJS [11]. NextJS представляет собой фреймворк для построения веб-приложений на базе React [17], обеспечивающий серверный рендеринг (SSR) и статическую генерацию страниц (SSG). Серверный рендеринг позволяет формировать HTML-код страниц на стороне сервера, что обеспечивает быструю первоначальную загрузку и оптимизацию для поисковых систем. NextJS поддерживает маршрутизацию на основе файловой системы, API-маршруты для создания серверных обработчиков и встроенную оптимизацию изображений и шрифтов. Сравнение NextJS с альтернативными фреймворками для веб-разработки представлено в таблице 1.

Таблица 1 — Сравнение NextJS с аналогами

Критерий	NextJS	React (SPA)	Gatsby	Nuxt.js
Серверный рендеринг (SSR)	Встроенный (getServerSideProps)	Требует дополнительной настройки	Статическая генерация (SSG)	Встроенный
Гибридный рендеринг	Поддержка SSR, SSG, ISR	Только клиентский рендеринг	Только SSG	Поддержка SSR и SSG
Маршрутизация	Файловая (App Router)	Требует react-router	Файловая на основе структуры	Файловая
Интеграция с API	Встроенные API Routes	Требует отдельный бэкэнд	Требует отдельный бэкэнд	Встроенные serverMiddleware
SEO-оптимизация	Отличная (SSR из коробки)	Требует дополнительной настройки	Отличная (SSG)	Отличная (SSR)
Экосистема	React-совместимый	Наибольшая экосистема	React-сообщество	Vue-сообщество
Производительность	Высокая (ISR, кэширование)	Зависит от реализации	Высокая (статические файлы)	Высокая
Сложность освоения	Средняя	Низкая	Средняя	Средняя (для React-разработчиков)

Для стилизации пользовательского интерфейса применён фреймворк TailwindCSS [19].

TailwindCSS — это утилитарный CSS-фреймворк, предоставляющий набор низкоуровневых классов для построения пользовательских интерфейсов без написания пользовательских CSS-правил.

Данный подход обеспечивает единообразие стилей, ускоряет процесс вёрстки и упрощает поддержку кодовой базы.

Библиотека компонентов PrimeReact [14] используется для создания интерактивных элементов пользовательского интерфейса.

PrimeReact предоставляет более восьмидесяти готовых компонентов, включая таблицы данных, формы, диалоговые окна, панели навигации и элементы отображения информации.

Компоненты библиотеки полностью совместимы с NextJS и поддерживают темы оформления, что обеспечивает единообразие визуального стиля всего приложения.

Изм.	Лист	№ документа	Подпись	Дата

BKP 09.02.07.22.4.2.3

Лист

16

HTTP-клиент Axios [1] применяется для выполнения запросов к серверному API из фронтенд-приложения.

Axios предоставляет удобный интерфейс для отправки HTTP-запросов, автоматическую сериализацию и десериализацию JSON-данных, обработку ошибок и возможность настройки интерцепторов для добавления заголовков аутентификации к каждому запросу.

Использование Axios позволяет централизовать логику взаимодействия с API и упрощает обработку типовых сценариев, таких как повторная отправка запроса при ошибке сети.

Бэкенд информационной системы построен на базе фреймворка NestJS [10].

NestJS — это прогрессивный фреймворк для создания серверных приложений на языке TypeScript [20], вдохновлённый архитектурой Angular.

Фреймворк предоставляет модульную структуру, встроенную поддержку внедрения зависимостей, декораторы для определения контроллеров, сервисов и провайдеров. NestJS поддерживает создание REST API, GraphQL-серверов и WebSocket-соединений, что делает его универсальным инструментом для построения серверной части.

PrismaORM [15] используется для взаимодействия бэкенда с базой данных PostgreSQL [13].

Prisma — это современная объектно-реляционная модель (ORM), обеспечивающая типобезопасный доступ к данным.

Схема данных описывается в декларативном файле Prisma Schema, на основе которого генерируется типизированный клиент для TypeScript.

PostgreSQL выбрана в качестве системы управления базами данных.

PostgreSQL — это открытая объектно-реляционная СУБД, поддерживающая сложные запросы, транзакции, внешние ключи, хранимые процедуры и расширения.

PostgreSQL обеспечивает надёжность хранения данных, поддержку ACID-транзакций и высокую производительность при работе с большими объёмами данных.

Мобильное приложение разработано с использованием платформы .NET MAUI [9] (Multi-platform App UI).

.NET MAUI — это кроссплатформенный фреймворк для создания нативных мобильных и десктопных приложений на единой кодовой базе.

Фреймворк позволяет разрабатывать приложения для Android, iOS, macOS и Windows, используя язык C# и единую систему проектирования пользовательского интерфейса.

Паттерн MVVM [8] (Model-View-ViewModel) применён в мобильном приложении для разделения пользовательского интерфейса и бизнес-логики.

Модель представляет данные и бизнес-правила, представление отвечает за отображение информации, а ViewModel выступает посредником, обеспечивая привязку данных и обработку команд пользователя.

Данный паттерн упрощает тестирование, повышает переиспользуемость кода и облегчает сопровождение приложения. Архитектура MVVM в контексте мобильного приложения показана на рисунке 5.

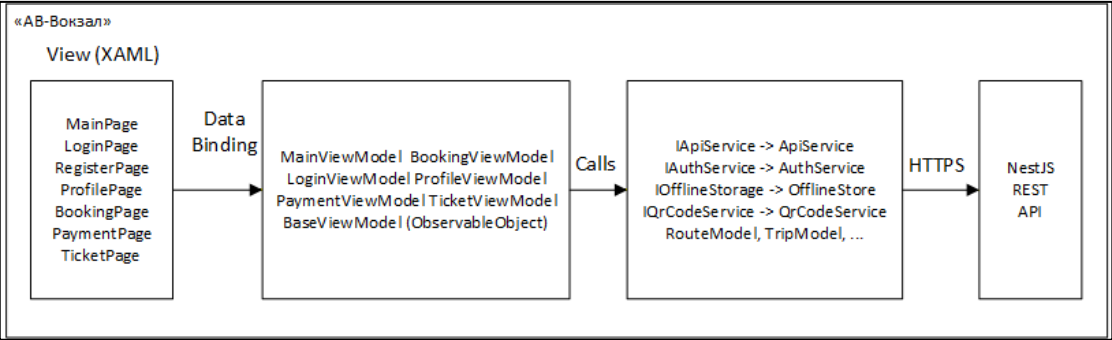


Рисунок 5 — Архитектура MVVM в контексте мобильного приложения «AB-Вокзал»

Совокупность выбранных технологий формирует технологический стек, обеспечивающий разработку полнофункциональной информационной системы с высоким уровнем производительности, безопасности и удобства сопровождения.

Выбранные технологии обеспечивают согласованность всех компонентов системы и единообразие подходов к разработке.

Сводная информация о технологическом стеке проекта представлена в приложении В.

1.4 Платформа развёртывания

Развёртывание информационной системы «АВ-Вокзал» организовано с использованием контейнеризации и облачных сервисов, что обеспечивает воспроизводимость окружения, масштабируемость и упрощение процесса деплоя. Выбор платформы развёртывания определялся требованиями к доступности системы, безопасности данных и удобству администрирования.

Контейнеризация компонентов системы выполнена с использованием технологии Docker [5]. Docker позволяет упаковать приложение и все его зависимости в изолированный контейнер, обеспечивая идентичное поведение приложения в средах разработки, тестирования и производства. Для каждого компонента системы — бэкенда, фронтенда и базы данных — определён собственный Dockerfile, описывающий процесс сборки образа. Dockerfile бэкенда включает установку зависимостей Node.js, компиляцию TypeScript [20]-кода и запуск NestJS [10]-приложения. Dockerfile фронтенда обеспечивает сборку NextJS [11]-приложения и запуск оптимизированной версии для продакшена.

Управление многоконтейнерным окружением осуществляется с помощью Docker Compose [6]. Файл docker-compose.yml описывает все сервисы системы, их зависимости, сети и тома для хранения данных. Docker Compose позволяет запустить всю инфраструктуру проекта одной командой, что значительно упрощает процесс развёртывания в локальной среде и на сервере. В конфигурации определены сервисы бэкенда, фронтенда и базы данных PostgreSQL [13], а также настроены переменные окружения для каждого сервиса. Фронтенд-приложение и бэкенд развёрнуты на платформе Vercel. Vercel — это облачная платформа, оптимизированная для размещения

NextJS-приложений, обеспечивающая автоматическую сборку, глобальную сеть доставки контента (CDN), HTTPS и автоматическое масштабирование. Интеграция Vercel с системой контроля версий позволяет настроить непрерывное развёртывание: при отправке изменений в основной репозиторий платформа автоматически собирает и публикует обновлённую версию приложения. Vercel поддерживает серверный рендеринг NextJS, что обеспечивает корректную работу SSR-страниц в продакшен-окружении. Бэкенд на базе NestJS также развёрнут на Vercel в виде серверных функций, что позволяет использовать единую платформу для обеих частей системы и упрощает управление инфраструктурой.

База данных PostgreSQL работает в выделенном Docker-контейнере с постоянным томом для хранения данных.

Использование тома обеспечивает сохранность данных при перезапуске контейнера и обновлении образа.

Переменные окружения используются для хранения конфиденциальной информации, такой как учётные данные базы данных, секретные ключи JWT и ключи платёжных систем.

Файл `.env.example` содержит шаблон переменных окружения с примерами значений, а реальный файл `.env` хранится в защищённом месте и не включается в систему контроля версий.

Процесс развёртывания включает следующие этапы: сборку Docker-образов для бэкенда и базы данных, запуск контейнеров через Docker Compose, настройку сетевого взаимодействия и проверку работоспособности сервисов.

Фронтенд и бэкенд развёртываются автоматически через интеграцию Vercel с репозиторием.

Таким образом, платформа развёртывания информационной системы «АВ-Вокзал» обеспечивает надёжную, масштабируемую и безопасную работу всех компонентов системы, используя современные практики контейнеризации, облачных вычислений и непрерывного развёртывания.

Вывод по теоретической части

Механика работы информационной системы охватывает полный цикл взаимодействия пассажиров с платформой. Система обеспечивает ролевую модель доступа: Администраторы управляют маршрутами и сотрудниками, водители получают информацию о рейсах и общаются в чате, а операторы мониторят задержки и корректируют расписание онлайн. Трёхзвенная клиент-серверная архитектура с REST API обеспечивает независимость компонентов и масштабируемость системы.

Анализ аналогов показал, что существующие платформы ориентированы преимущественно на пассажиров и не предоставляют комплексных инструментов для управления автовокзалом. Разрабатываемая система восполняет этот пробел, объединяя пассажирские сервисы с инструментами оперативного управления: встроенным чатом сотрудников и онлайн-отслеживанием задержек.

Технологический стек проекта включает NextJS для фронтенда, NestJS и PrismaORM для бэкенда, PostgreSQL для хранения данных и .NET MAUI с CommunityToolkit.MVVM для мобильного приложения. Каждая технология выбрана на основе анализа альтернативных решений и обеспечивает производительность, типобезопасность, масштабируемость и удобство сопровождения. NextJS предоставляет серверный рендеринг, NestJS — модульную архитектуру, PrismaORM гарантирует типобезопасную работу с базой, а .NET MAUI позволяет создать кроссплатформенное приложение с единой кодовой базой.

Платформа развёртывания основана на контейнеризации с использованием Docker и Docker Compose, облачном хостинге Vercel для фронтенда и бэкенда, а также выделенном сервере для базы данных. Подобная архитектура обеспечивает воспроизводимость окружения, автоматическое масштабирование, непрерывное развёртывание и защищённое хранение данных.

					<i>ВКР 09.02.07.22.4.2.3</i>	Лист
						21
Изм.	Лист	№ документа	Подпись	Дата		

2 ПРАКТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ

2.1 Проектирование системы

Проектирование информационной системы «АВ-Вокзал» начинается с определения архитектурных решений, которые обеспечивают масштабируемость, безопасность и удобство сопровождения. Система спроектирована по трёхзвенной клиент-серверной архитектуре, разделяющей презентационный слой (фронтенд и мобильное приложение), слой бизнес-логики (бэкенд API) и слой хранения данных (PostgreSQL). Подобное разделение позволяет независимо развивать каждый компонент, заменять технологии на отдельных уровнях и масштабировать систему горизонтально при росте нагрузки.

На этапе проектирования формируется перечень функциональных требований, определяющий возможности системы для каждой роли пользователей. Пассажиры получают возможность поиска маршрутов, бронирования билетов, онлайн-оплаты, управления избранными маршрутами и хранения электронных билетов. Администраторы управляют маршрутами, рейсами и учётными записями сотрудников. Водители просматривают назначенные рейсы и общаются с операторами через встроенный чат. Операторы мониторят задержки, корректируют расписание и управляют рейсами. Карта вариантов использования (Use Case Diagram) информационной системы представлена на рисунке 6.

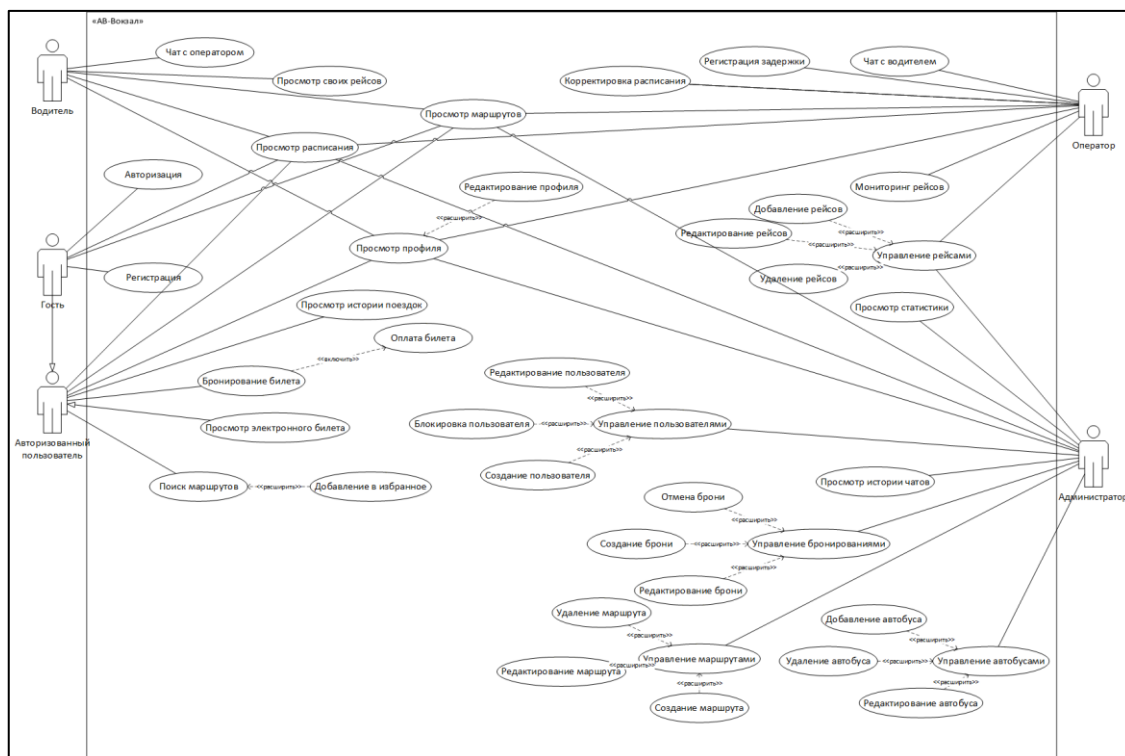


Рисунок 6 — Диаграмма вариантов использования ИС «АВ-Вокзал»

Архитектура системы проектируется с использованием паттерна REST API [18], который определяет стандартные HTTP-методы для выполнения операций над ресурсами.

Каждый ресурс системы — пользователи, маршруты, рейсы, бронирования, билеты, платежи, сообщения чата и задержки — имеет собственный набор конечных точек (endpoints) с чётко определёнными контрактами запросов и ответов.

Формат запросов и ответов унифицирован: все данные передаются в формате JSON, ошибки возвращаются с соответствующими HTTP-кодами статуса. Проектирование системы включает ролевую модель доступа с четырьмя ролями. Проверка прав доступа выполняется на двух уровнях: на стороне сервера через JWT-гарды NestJS [10] и на стороне клиента через middleware NextJS [11]. Схема ролевой модели доступа представлена в таблице 2.

Таблица 2 — Схема ролевой модели доступа

Функционал	Ролевая модель				
	Гость	Пользователь	Водитель	Оператор	Администратор
Веб-сайт (публичный)	Да	Да	Да	Да	Да
Личный кабинет	Нет	Да	Да	Да	Да
Панель оператора	Нет	Нет	Нет	Да	Нет
Панель водителя	Нет	Нет	Да	Нет	Нет
Панель администратора	Нет	Нет	Нет	Нет	Да
Управление маршрутами	Нет	Нет	Нет	Нет	Да
Управление рейсами	Нет	Нет	Нет	Да	Да
Управление пользователями	Нет	Нет	Нет	Нет	Да
Чат водителя и оператора	Нет	Нет	Да	Да	Нет
Просмотр истории чатов	Нет	Нет	Нет	Нет	Да

Процесс аутентификации проектируется на основе JSON Web Tokens (JWT) [7].

При регистрации или входе пользователь получает токен доступа, содержащий идентификатор пользователя, адрес электронной почты и роль.

Токен передаётся в заголовке Authorization каждого последующего запроса и проверяется сервером. Диаграмма последовательности процесса аутентификации показана на рисунке 7.

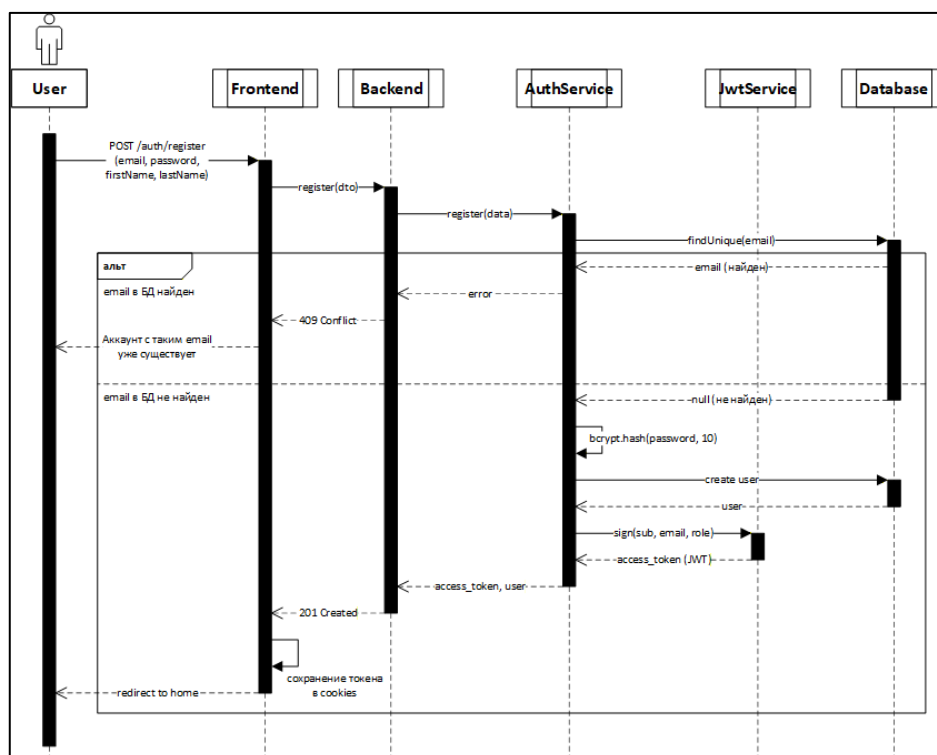


Рисунок 7 — Диаграмма последовательности процесса аутентификации

Проектирование пользовательских интерфейсов выполняется с учётом принципов отзывчивого дизайна, обеспечивающих корректное отображение на устройствах с различными размерами экранов.

Веб-приложение включает главную страницу, каталог маршрутов, страницу бронирования, личный кабинет, страницу оплаты, а также специализированные панели для администраторов, водителей и операторов.

Каждый интерфейс проектируется с использованием компонентной библиотеки PrimeReact [14], что обеспечивает единообразие визуального стиля. Проектирование системы чата между водителями и операторами основано на модели хранения сообщений в базе данных с флагом прочтения.

Проектирование системы отслеживания задержек включает создание записей о задержках с привязкой к конкретному рейсу.

При создании записи о задержке статус рейса автоматически изменяется на DELAYED, что инициирует обновление информации на всех клиентских устройствах.

Оператор может корректировать время рейса, после чего статус возвращается в SCHEDULED.

Проектирование системы бронирования и оплаты определяет последовательность операций: выбор маршрута, создание бронирования, оплата, подтверждение, генерация электронного билета с QR-кодом.

Бронирование имеет ограниченный срок действия, после истечения которого неоплаченная бронь аннулируется.

При создании бронирования выполняется проверка доступности мест: система загружает все активные бронирования рейса и сравнивает с вместимостью автобуса. Диаграмма последовательности процесса бронирования и оплаты показана на рисунке 8.

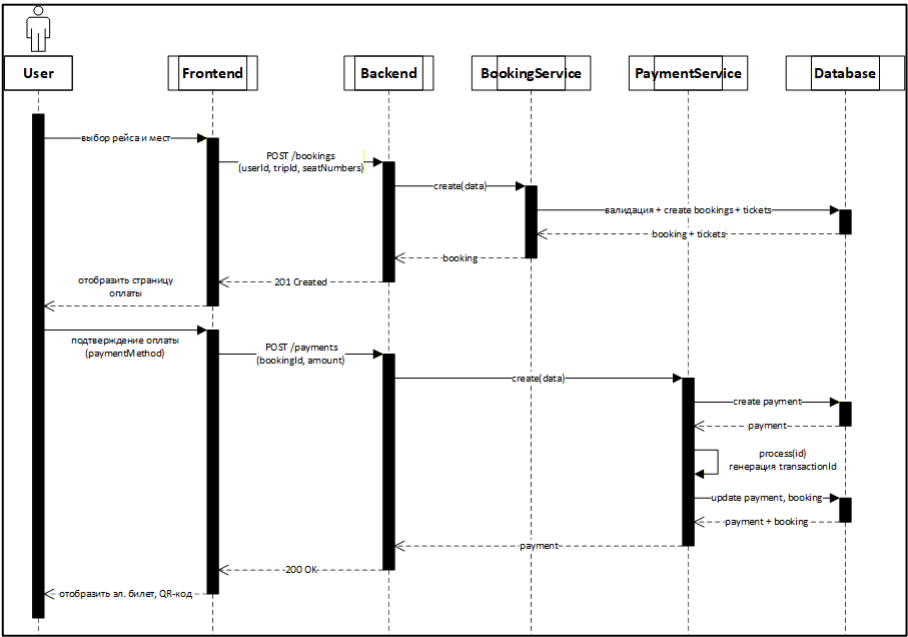


Рисунок 8 — Диаграмма последовательности процесса бронирования и оплаты

Проектирование базы данных выполняется с использованием PrismaORM, который обеспечивает типобезопасное описание схемы данных. Схема определяет модели, поля, типы данных, связи и ограничения целостности. Нормализация до третьей нормальной формы исключает избыточность данных и обеспечивает целостность при операциях обновления и удаления. Каждая модель имеет автоматические поля createdAt и updatedAt, которые заполняются при создании и обновлении записей.

2.2 Разработка базы данных

Разработка базы данных информационной системы «АВ-Вокзал» осуществляется с использованием системы управления базами данных PostgreSQL [13] и объектно-реляционной модели PrismaORM [15]. Prisma Schema определяет структуру всех таблиц, типы данных, связи между моделями и ограничения целостности. Схема данных служит единственным источником истины для структуры базы и используется для автоматической генерации миграций и типизированного клиента TypeScript. PostgreSQL обеспечивает надёжное хранение данных, поддержку ACID-транзакций и высокую производительность при работе с большими объёмами данных.

Центральной моделью базы данных является модель User, представляющая пользователей системы. Модель содержит поля для хранения адреса электронной почты, хешированного пароля, имени, фамилии, номера телефона, роли и статуса активности. Поле email имеет уникальный индекс, предотвращающий регистрацию дубликатов. Поле role использует перечисление UserRole с четырьмя значениями: USER, ADMIN, DRIVER, OPERATOR. Поле isActive обеспечивает возможность мягкого удаления пользователей без потери связанных данных — вместо физического удаления записи флаг устанавливается в false, и пользователь исключается из выборок активных пользователей. Модель User связана с моделями Booking, Favorite, Trip, Chat и Message через отношения один-ко-многим. Структура модели User представлена в таблице 3.

Таблица 3 — Структура модели User

Поле	Тип данных	Атрибуты	Описание
id	String (UUID)	@id @default(uuid())	Уникальный идентификатор пользователя, генерируется автоматически
email	String	@unique	Адрес электронной почты (уникальный, используется для входа)
password	String	—	Хэш пароля пользователя (bcrypt)
firstName	String	—	Имя пользователя
lastName	String	—	Фамилия пользователя
phone	String?	—	Номер телефона (необязательное поле)
role	UserRole	@default(USER)	Роль пользователя: USER, ADMIN, DRIVER, OPERATOR

<i>isActive</i>	Boolean	@default(true)	Флаг активности учётной записи
<i>createdAt</i>	DateTime	@default(now())	Дата и время создания записи
<i>updatedAt</i>	DateTime	@updatedAt	Дата и время последнего обновления записи

Модель *Route* описывает автобусные маршруты и включает поля для наименования, описания, пункта отправления, пункта назначения, расстояния, длительности и стоимости.

Поле *name* хранит краткое наименование маршрута, поле *price* хранит стоимость в формате *Decimal* с точностью до двух знаков после запятой.

Поле *isActive* позволяет мягко удалять маршруты без удаления связанных рейсов и избранных записей.

Модель *Trip* представляет конкретные рейсы по маршрутам и содержит поля для идентификатора маршрута, времени отправления, времени прибытия, статуса рейса, номера автобуса, идентификатора водителя и идентификатора оператора. Поле *departureTime* хранит дату и время отправления, *arrivalTime* — дату и время прибытия. Поле *status* использует перечисление *TripStatus* со значениями: *SCHEDULED* (запланирован), *IN_PROGRESS* (в пути), *COMPLETED* (завершён), *CANCELLED* (отменён), *DELAYED* (задерживается).

Модель *Booking* хранит информацию о бронированиях и включает поля для идентификатора пользователя, идентификатора рейса, количества мест, общей стоимости, статуса бронирования, имени пассажира и номера телефона пассажира. Поле *seats* хранит количество забронированных мест. Поле *totalPrice* содержит общую стоимость бронирования, вычисляемую как произведение стоимости маршрута на количество мест. Поле *status* использует перечисление *BookingStatus* со значениями: *PENDING* (ожидает оплаты), *CONFIRMED* (подтверждено), *CANCELLED* (отменено), *COMPLETED* (завершено). Номера выбранных мест вынесены в отдельную модель *BookingSeat*, что обеспечивает приведение схемы к третьей нормальной форме. Модель *Booking* связана с моделями *User*, *Trip*, *Payment*, *Ticket* и *BookingSeat*.

					ВКР 09.02.07.22.4.2.3	Лист
						28
Изм.	Лист	№ документа	Подпись	Дата		

Модель BookingSeat хранит информацию о выбранных местах в бронировании и содержит поля для идентификатора бронирования и номера места. Составной уникальный индекс по полям bookingId и seatNumber гарантирует, что одно место не может быть забронировано дважды в рамках одного бронирования. Вынесение номеров мест в отдельную модель обеспечивает нормализацию схемы данных до третьей нормальной формы. Модель BookingSeat связана с моделью Booking через внешний ключ bookingId с каскадным удалением — при отмене бронирования записи о местах удаляются автоматически.

Модель Ticket представляет электронные билеты и содержит поля для идентификатора бронирования, идентификатора рейса, номера места, QR-кода, флага использования и времени использования. Поле qrCode хранит уникальный идентификатор билета в формате TICKET-`<hex>`, генерируемый с использованием криптографически стойкого генератора случайных чисел. Поле seatNumber указывает номер места в автобусе. Поле isUsed отслеживает факт использования билета при посадке, поле usedAt фиксирует время использования. Модель Ticket связана с моделями Booking и Trip.

Модель Payment хранит информацию об оплатах и включает поля для идентификатора бронирования, суммы, статуса оплаты, метода оплаты, идентификатора транзакции и времени оплаты. Поле bookingId имеет уникальный индекс, обеспечивающий наличие не более одной оплаты на бронирование. Поле status использует перечисление PaymentStatus со значениями: PENDING (ожидает), COMPLETED (завершено), FAILED (ошибка), REFUNDED (возврат). Поле transactionId генерируется автоматически при обработке оплаты и содержит метку времени и случайную строку для уникальности. Поле paidAt фиксирует время успешной оплаты.

Модель Favorite обеспечивает хранение избранных маршрутов пользователей и содержит поля для идентификатора пользователя и идентификатора маршрута. Составной уникальный индекс по userId и routeId гарантирует, что пользователь не может добавить один маршрут в избранное

дважды. Модель связана с User и Route через отношения многие-ко-многим с дополнительной таблицей.

Модель Delay хранит информацию о задержках рейсов и включает поля для идентификатора рейса, причины задержки и количества минут задержки. Поле tripId связывает задержку с конкретным рейсом. Поле reason содержит текстовое описание причины задержки. Поле delayMinutes хранит количество минут задержки в целочисленном формате. При создании записи о задержке статус связанного рейса автоматически изменяется на DELAYED.

Модель Bus (автобус) хранит информацию о транспортных средствах и включает поля для государственного номера, модели и общего количества мест. Поле plateNumber имеет уникальный индекс, исключающий дублирование записей об одном автобусе. Поле model хранит название модели автобуса, поле totalSeats определяет вместимость транспортного средства. Модель Bus связана с моделью Trip через внешний ключ busId, что позволяет закреплять конкретный автобус за рейсом.

Модель Chat представляет чат между водителем и оператором и содержит поля для идентификатора водителя, идентификатора оператора и статуса чата. Поле status использует перечисление ChatStatus со значениями WAITING (ожидает оператора), ACTIVE (активный чат) и CLOSED (закрыт). Для контроля активности соединения поле lastHeartbeatAt фиксирует время последнего сигнала от водителя, при отсутствии сигнала в течение заданного таймаута чат автоматически закрывается. Модель Chat связана с моделями User и Message.

Модель Message представляет сообщение в чате и содержит поля для идентификатора чата, идентификатора отправителя, его роли и текста сообщения. Поле senderRole использует перечисление MessageSenderRole со значениями DRIVER, OPERATOR и ADMIN, что определяет, кто отправил сообщение. Поле text хранит содержимое сообщения в формате Text произвольной длины. Сообщения упорядочиваются по времени создания, что

обеспечивает хронологический порядок переписки. Модель Message связана с моделями Chat и User.

Для обеспечения целостности данных используются внешние ключи на уровне базы данных. PrismaORM автоматически создаёт ограничения внешних ключей при применении миграций, что гарантирует ссылочную целостность на уровне СУБД. Каскадное удаление настраивается selectively: при удалении пользователя его бронирования сохраняются, однако при удалении маршрута связанные избранные записи удаляются каскадно. Связь между Trip и Route защищает от удаления маршрута, пока существуют связанные рейсы.

Индексы базы данных создаются для полей, участвующих в частых запросах: email пользователя, идентификатор маршрута в рейсах, идентификатор пользователя в бронированиях. Составной уникальный индекс на модели Favorite обеспечивает эффективность проверки избранного. Миграции базы данных управляются через Prisma Migrate. Каждая миграция представляет собой SQL-скрипт, описывающий изменения схемы данных. Prisma отслеживает применённые миграции в системной таблице `_prisma_migrations` и обеспечивает возможность отката к предыдущим версиям схемы. Процесс применения миграций автоматизирован и выполняется при развёртывании приложения. Начальная миграция создаёт все таблицы, индексы и перечисления, определённые в Prisma Schema.

2.3 Написание API

Разработка серверного API информационной системы «АВ-Вокзал» выполняется на базе фреймворка NestJS [10] с использованием языка TypeScript [20]. NestJS обеспечивает модульную архитектуру, внедрение зависимостей, декораторы для определения контроллеров и сервисов, а также встроенные механизмы аутентификации, валидации и обработки ошибок. API построено по принципу REST и предоставляет конечные точки для всех ресурсов системы.

					<i>ВКР 09.02.07.22.4.2.3</i>	Лист
Изм.	Лист	№ документа	Подпись	Дата		31

Модуль аутентификации отвечает за регистрацию новых пользователей, вход в систему и валидацию токенов доступа. При регистрации выполняется проверка уникальности адреса электронной почты, хеширование пароля с использованием библиотеки bcrypt [2] с фактором стоимости 10, создание записи пользователя в базе данных и генерация JWT-токена. Код метода регистрации приведён в листинге 1.

Листинг 1 — Метод регистрации

```

1: async register(data: RegisterDto): Promise<AuthResponse> {
2:   const existingUser = await this.prisma.user.findUnique({
3:     where: { email: data.email },
4:   });
5:   const hashedPassword =
6:     await bcrypt.hash(data.password, 10);
7:   const user = await this.prisma.user.create({
8:     data: {
9:       email: data.email,
10:      password: hashedPassword,
11:      firstName: data.firstName,
12:      lastName: data.lastName,
13:      phone: data.phone || null,
14:      role: 'USER',
15:    },
16:   });
17:   const payload: TokenPayload = {
18:     sub: user.id,
19:     email: user.email,
20:     role: user.role,
21:   };
22:   const access_token = this.jwtService.sign(payload);
23:   return {
24:     access_token,
25:     user: {
26:       id: user.id,
27:       email: user.email,
28:       firstName: user.firstName,
29:       lastName: user.lastName,
30:       role: user.role,
31:     },
32:   };
33: }

```

Метод входа в систему выполняет поиск пользователя по адресу электронной почты, проверку активности аккаунта, верификацию пароля через bcrypt.compare и генерацию JWT-токена. При неверном пароле или отсутствии пользователя возвращается ошибка UnauthorizedException с универсальным сообщением, не раскрывающим причину отказа. Метод

дополнительно проверяет, не заблокирован ли аккаунт пользователя, и при обнаружении блокировки возвращает соответствующую ошибку.

JWT-стратегия использует Passport.js [12] для извлечения данных пользователя из токена. Метод validate принимает полезную нагрузку токена и возвращает объект с идентификатором пользователя, адресом электронной почты и ролью. JWT-гард расширяет стандартный AuthGuard и переопределяет метод handleRequest для возврата null вместо выброса исключения при ошибке аутентификации, что позволяет контроллерам самостоятельно решать, как обрабатывать отсутствующего пользователя.

Модуль маршрутов предоставляет конечные точки для получения списка маршрутов, поиска по пунктам отправления и назначения, создания, обновления и удаления маршрутов. Поиск маршрутов выполняется с использованием case-insensitive частичного совпадения по полям origin и destination. Метод удаления выполняет мягкое удаление, устанавливая флаг isActive в false.

Модуль рейсов обеспечивает управление рейсами: получение списка рейсов, фильтрацию по маршруту и водителю, получение рейсов с задержками, обновление времени рейса и расчёт доступных мест. Метод расчёта доступных мест загружает рейс со связанными бронированиями в статусах CONFIRMED и PENDING, суммирует забронированные места и возвращает разницу с общим количеством мест в автобусе.

Модуль бронирования реализует создание бронирований, подтверждение, отмену и получение списка бронирований пользователя. При создании бронирования выполняется поиск рейса с связанным маршрутом, вычисление общей стоимости как произведения цены маршрута на количество мест и создание записи бронирования со статусом PENDING. Система проверяет наличие доступных мест перед созданием бронирования, чтобы избежать превышения вместимости автобуса.

Модуль билетов обеспечивает создание электронных билетов с уникальными QR-кодами, валидацию билетов и отметку об использовании.

					<i>БКР 09.02.07.22.4.2.3</i>	Лист
Изм.	Лист	№ документа	Подпись	Дата		33

При создании билета генерируется уникальный QR-код в формате TICKET-`<hex>` с использованием криптографически стойкого генератора случайных чисел `randomBytes`. Сгенерированный QR-код гарантирует уникальность каждого билета и защищён от подделки за счёт криптографической стойкости.

Модуль платежей обрабатывает создание платежей, обработку оплаты и возврат средств. При обработке платежа генерируется уникальный идентификатор транзакции в формате TXN-`<timestamp>`-`<random>`, статус изменяется на COMPLETED и фиксируется время оплаты. Идентификатор транзакции строится из временной метки и случайной строки, что исключает коллизии при параллельных платежах.

Модуль чата обеспечивает отправку сообщений, получение истории переписки, отметку сообщений как прочитанных и подсчёт непрочитанных сообщений. При отправке сообщения выполняется запись в базу данных с привязкой к идентификатору водителя. Логирование отправки сообщения с первыми 50 символами содержимого упрощает отладку при возникновении проблем с чатом.

Модуль задержек создаёт записи о задержках рейсов и автоматически обновляет статус рейса на DELAYED. При создании записи о задержке выполняется обновление статуса рейса и создание записи в таблице `delays` с указанием причины и количества минут задержки. Автоматическое обновление статуса рейса гарантирует, что информация о задержке немедленно становится доступна всем заинтересованным сторонам.

Модуль избранного позволяет пользователям добавлять маршруты в избранное, удалять из избранного и проверять статус избранного. При добавлении маршрута выполняется проверка на существование записи — если маршрут уже в избранном, возвращается существующая запись без создания дубликата.

Модуль пользователей обеспечивает получение списка пользователей с фильтрацией по роли, создание, обновление и удаление пользователей. Метод

получения списка водителей возвращает только активных пользователей с ролью DRIVER, исключая конфиденциальные поля (пароль, дата создания).

Глобальный interceptor логирования регистрирует каждый HTTP-запрос с указанием метода, URL, кода статуса и времени выполнения. Interceptor подключается на уровне приложения и применяется ко всем конечным точкам API. Настройка CORS разрешает запросы с доменов, указанных в переменной окружения CORS_ORIGINS, и поддерживает передачу учётных данных.

Валидация входящих данных выполняется с использованием DTO (Data Transfer Objects) и декораторов class-validator [3]. NestJS автоматически валидирует тело запроса и возвращает ошибку BadRequestException при нарушении правил валидации. Ролевые гарды проверяют роль пользователя через декоратор @Roles() и сравнивают её с ролью из JWT-токена. Структура модулей API отражена в приложении Г.

2.4 Разработка сайта

Разработка веб-приложения информационной системы «АВ-Вокзал» выполняется на базе фреймворка NextJS [11] с использованием языка TypeScript [20]. NextJS обеспечивает серверный рендеринг, маршрутизацию на основе файловой системы и оптимизацию производительности. Пользовательский интерфейс строится с использованием библиотеки компонентов PrimeReact [14] и фреймворка TailwindCSS [19] для стилизации.

Клиентский слой взаимодействия с API реализован через Axios instance с базовым URL, настраиваемым через переменную окружения NEXT_PUBLIC_API_URL. Axios interceptor автоматически добавляет заголовок Authorization с JWT-токеном из localStorage к каждому запросу, что исключает необходимость ручной передачи токена в каждом вызове API.

Слой аутентификации на клиенте реализован через React Context, предоставляющий глобальное состояние аутентификации для всего приложения. Контекст хранит данные пользователя, токен доступа, флаги загрузки и аутентификации. Метод login выполняет запрос к API, сохраняет

токен в cookies и localStorage, обновляет состояние контекста. При неудачном запросе метод возвращает объект с флагом success: false и текстом ошибки.

Middleware NextJS (файл proxy.ts) выполняет проверку аутентификации и ролевого доступа на стороне сервера при каждом запросе к защищённым маршрутам. Middleware извлекает токен и данные пользователя из cookies, определяет роль и сравнивает её с требуемой для доступа к запрашиваемому маршруту. При отсутствии токена пользователь перенаправляется на страницу входа с сохранением текущего пути для возврата. При несоответствии роли пользователь перенаправляется на главную страницу. Код middleware приведён в листинге 2.

Листинг 2 — Middleware

```
1: export function proxy(request: NextRequest) {
2:   const token = request.cookies.get('access_token')?.value;
3:   const userJson = request.cookies.get('user')?.value;
4:   const pathname = request.nextUrl.pathname;
5:   let userRole: string | null = null;
6:   if (userJson) {
7:     try {
8:       const user = JSON.parse(userJson);
9:       userRole = user?.role || null;
10:    } catch {userRole = null; }}
11:   const requiresAuth =
12:     protectedRoutes.some(route
13:       => pathname.startsWith(route));
14:   if (requiresAuth && !token) {
15:     const loginUrl = new URL('/auth/login', request.url);
16:     loginUrl.searchParams.set('redirect', pathname);
17:     return NextResponse.redirect(loginUrl); }
18:   const requiresAdmin =
19:     adminRoutes.some(route => pathname.startsWith(route));
20:   if (requiresAdmin && userRole !== 'ADMIN') {
21:     return NextResponse.redirect(new URL('/', request.url));}
22:   const requiresDriver =
23:     driverRoutes.some(route => pathname.startsWith(route));
24:   if (requiresDriver && userRole !== 'DRIVER') {
25:     return NextResponse.redirect(new URL('/', request.url));}
26:   const requiresOperator =
27:     operatorRoutes.some(route => pathname.startsWith(route));
28:   if (requiresOperator && userRole !== 'OPERATOR') {
29:     return NextResponse.redirect(new URL('/', request.url));}
30:   return NextResponse.next(); }
```

Страница каталога маршрутов отображает список доступных направлений в виде карточек с информацией о пунктах отправления и

назначения, расстоянии, длительности и стоимости. Каждая карточка содержит кнопку бронирования, при нажатии на которую проверяется аутентификация пользователя. Для неавторизованных пользователей выполняется перенаправление на страницу входа с сохранением текущего маршрута для возврата после авторизации. Каталог маршрутов представлен на рисунке 9.

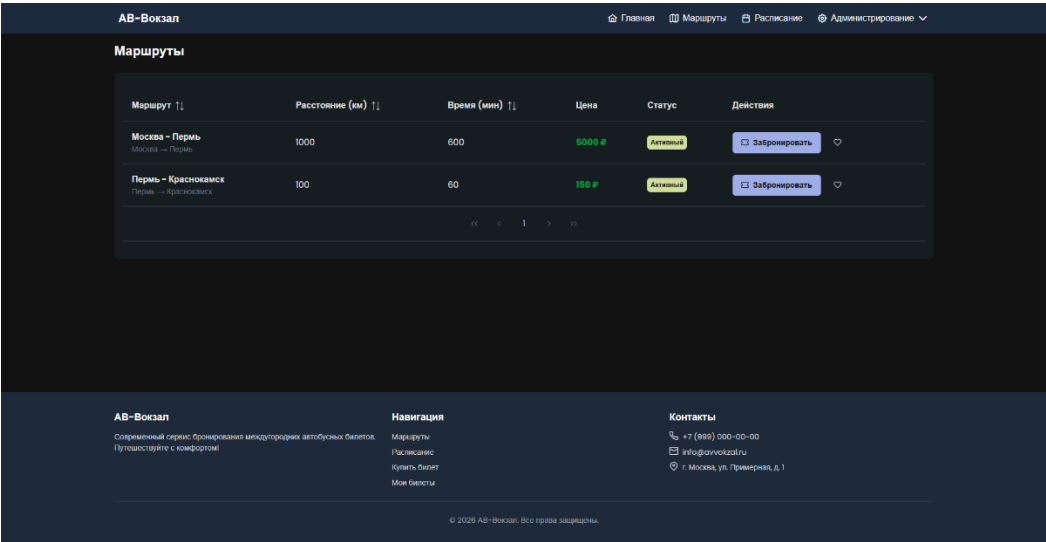


Рисунок 9 — Страница каталога маршрутов

Страница бронирования билета представляет собой диалоговое окно, в котором пользователь выбирает конкретный рейс из списка доступных, указывает количество мест, вводит имя пассажира и номер телефона. При оформлении бронирования система запрашивает количество доступных мест через API и сравнивает с запрошенным количеством — если мест недостаточно, отображается предупреждение. После успешного создания бронирования пользователь получает подтверждение с информацией о рейсе и общей стоимости. Диалоговое окно реализовано с использованием PrimeReact Dialog и содержит форму с валидацией полей. Страница бронирования показана на рисунке 10.

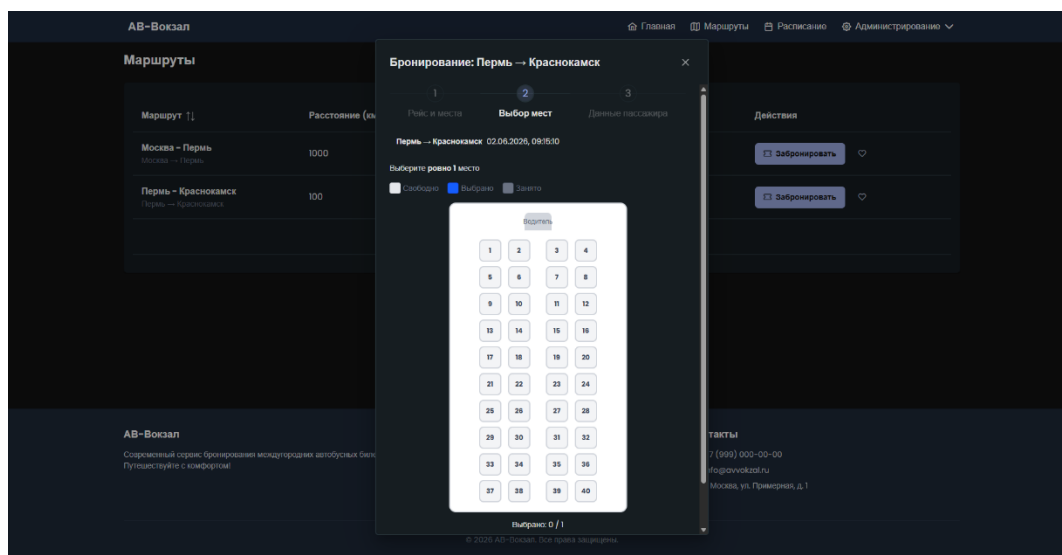


Рисунок 10 — Страница бронирования

Страница оплаты отображает детали созданного бронирования: маршрут, дату, количество мест и общую стоимость. Пользователь выбирает метод оплаты, после чего система создаёт запись платежа, обрабатывает оплату через API и подтверждает бронирование. При успешной оплате пользователь получает уведомление о подтверждении. Страница оплаты показана на рисунке 11.

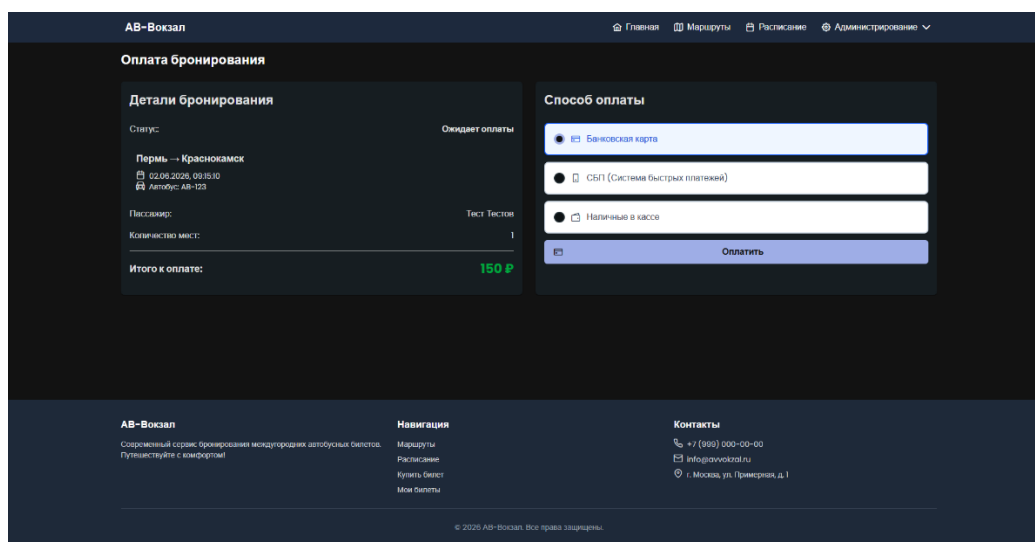


Рисунок 11 — Страница оплаты

Страница расписания рейсов отображает таблицу со всеми рейсами, их статусами (запланирован, задержан, в пути, завершён, отменён) и цветовой индикацией. Компонент использует PrimeReact DataTable с поддержкой сортировки, фильтрации по дате и маршруту. Пользователь может

отслеживать актуальное состояние каждого рейса в реальном времени. Страница расписания показана на рисунке 12.

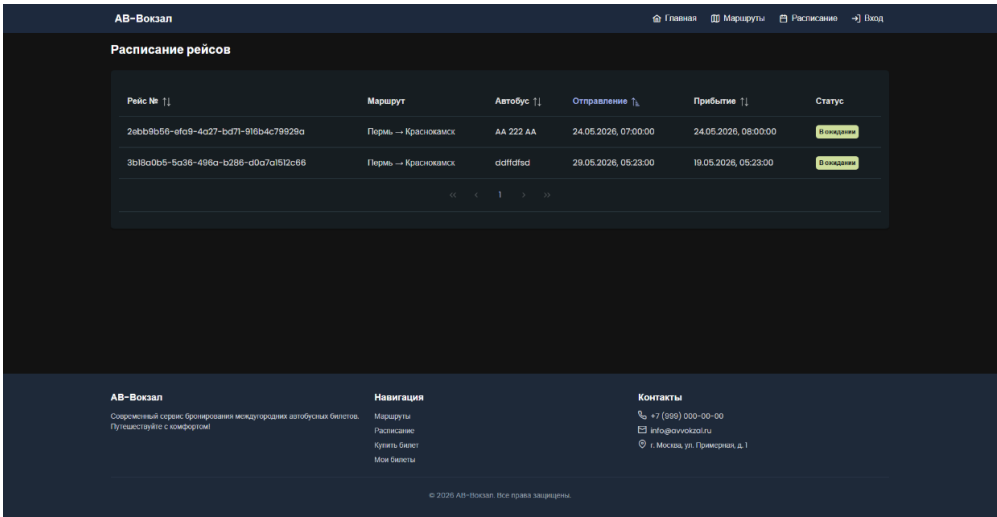


Рисунок 12 — Страница расписания

Панель администратора включает разделы управления маршрутами, рейсами и учётными записями пользователей. Страница управления пользователями отображает таблицу с возможностью поиска, создания, редактирования и удаления записей. Доступ к панели ограничен пользователями с ролью ADMIN на уровне middleware. Панель администратора показана на рисунке 13.

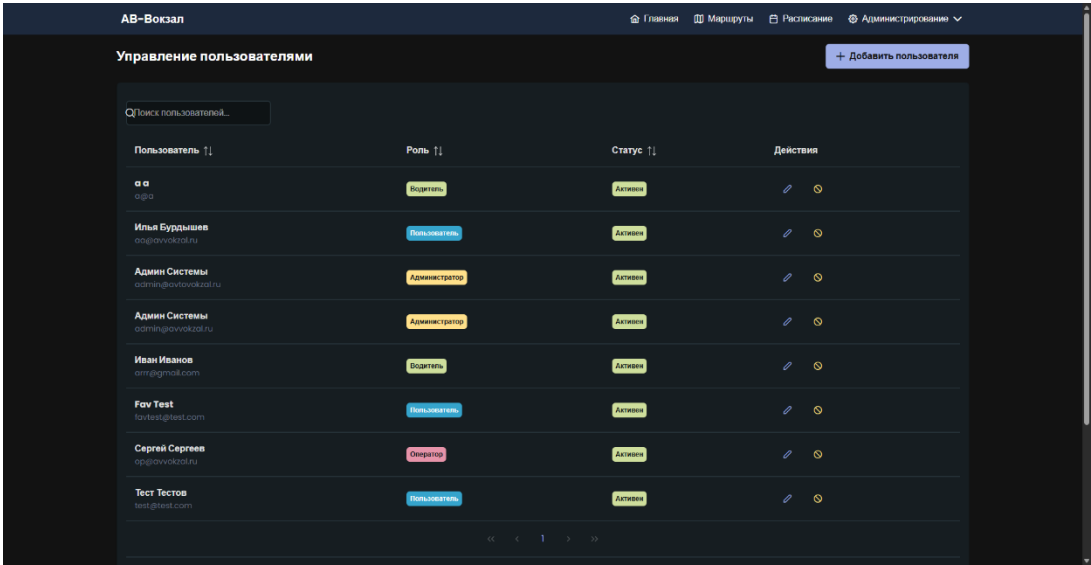


Рисунок 13 — Панель управления пользователями

Панель оператора обеспечивает мониторинг активных рейсов, создание записей о задержках, корректировку времени отправления и прибытия, а также создание новых рейсов. При создании задержки оператор указывает

причину и длительность, после чего информация автоматически обновляется на всех клиентских устройствах. Панель оператора показана на рисунке 14.

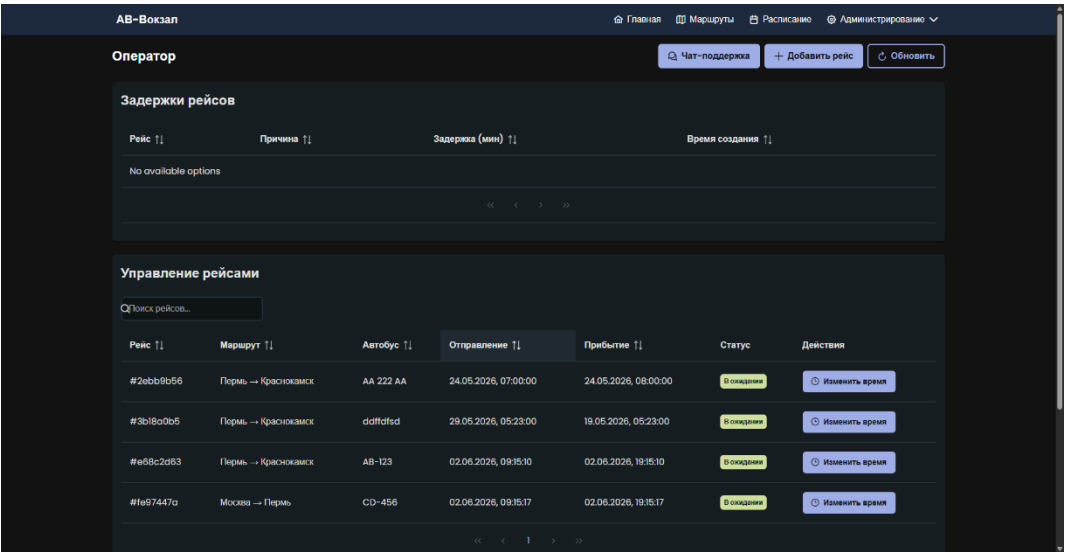


Рисунок 14 — Панель управления оператора

Панель водителя отображает назначенные рейсы с информацией о маршруте, времени и статусе. В нижней части страницы расположен интерфейс чата с оператором, содержащий поле ввода, кнопку отправки и историю переписки. Обновление чата происходит через интервальный опрос сервера для получения новых сообщений. Панель водителя показана на рисунке 15.

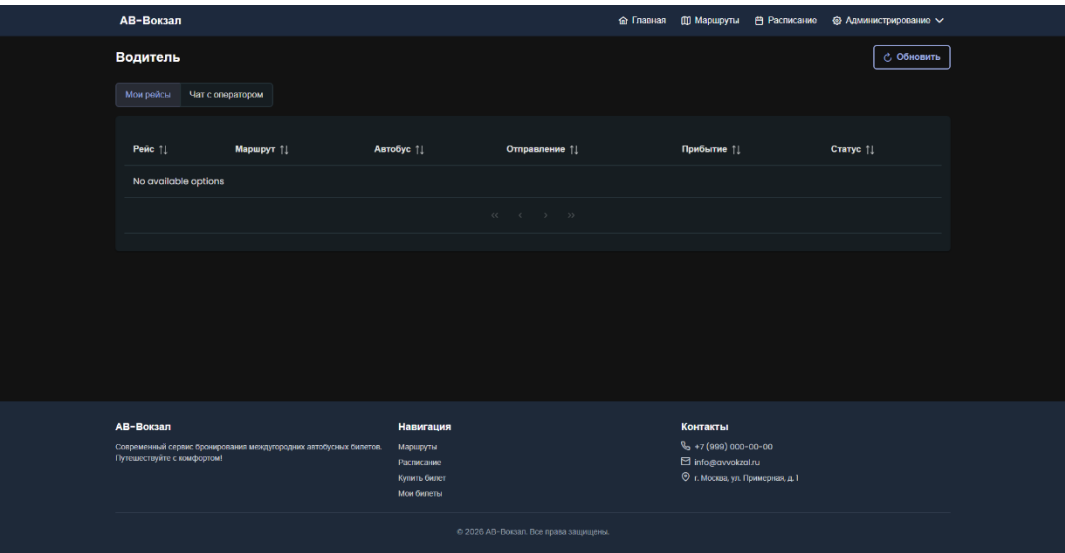


Рисунок 15 — Панель управления водителя

Страница профиля пользователя включает вкладки с информацией о бронированиях, электронных билетах и избранных маршрутах. Каждый билет отображается с QR-кодом, который можно предъявить при посадке.

Навигация между разделами профиля реализована через PrimeReact TabView. Страница профиля показана на рисунке 16.

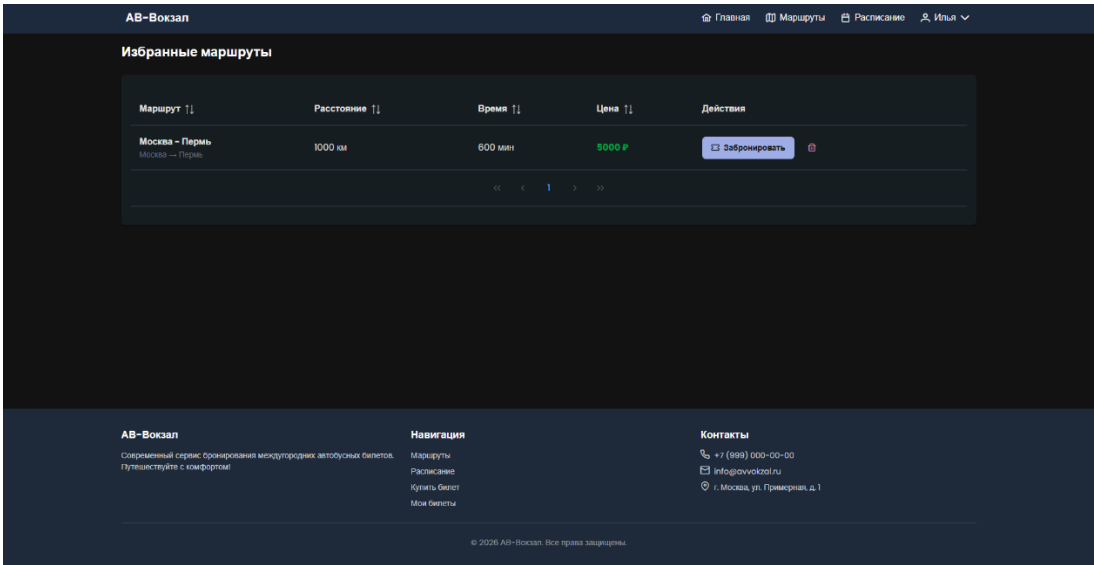


Рисунок 16 — Страница избранных маршрутов

Страница электронного билета отображает детальную информацию о билете: номер билета, маршрут, дату и время отправления, номер места, имя пассажира и QR-код. QR-код содержит зашифрованную информацию о рейсе, месте и идентификаторе билета для проверки подлинности при посадке. Подлинность билета проверяется на стороне сервера при сканировании QR-кода контролёром. Страница электронного билета показана на рисунке 17.

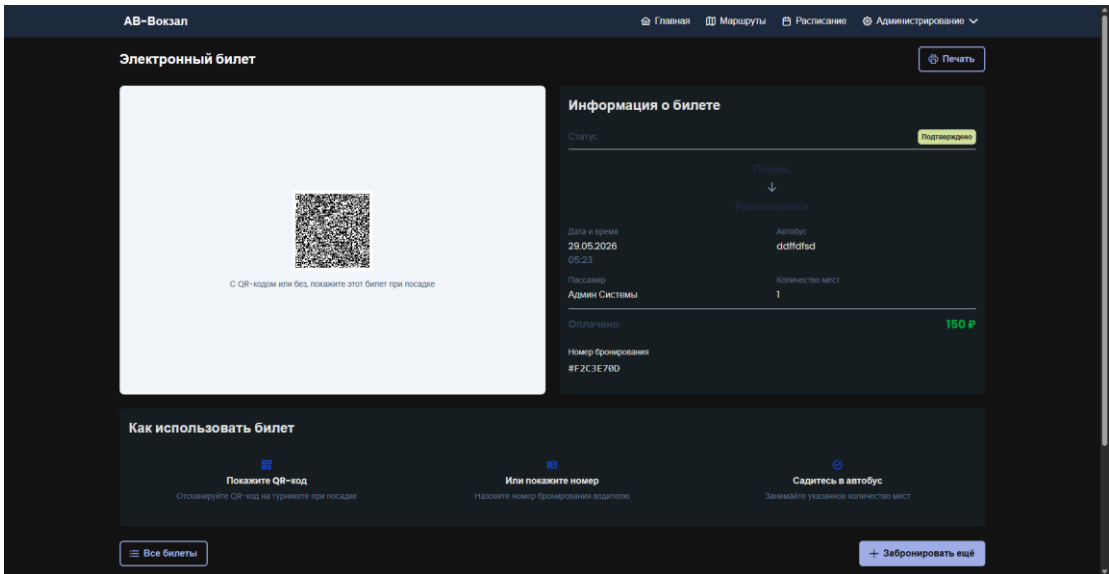


Рисунок 17 — Страница электронного билета

Навигационная панель сайта адаптируется в зависимости от роли пользователя: для администраторов отображается ссылка на панель

управления, для водителей — на панель водителя, для операторов — на панель оператора. Выпадающие меню управляются через состояние активного элемента, предотвращая одновременное открытие нескольких меню.

2.5 Разработка мобильного приложения

Мобильное приложение разработано на .NET MAUI [9] с паттерном MVVM [8] и CommunityToolkitMVVM [4] для Android и iOS. Приложение ориентировано на пассажиров и взаимодействует с тем же REST API [18], что и веб-версия, обеспечивая единообразие данных.

ViewModel наследуют BaseViewModel с общими свойствами (IsBusy, ErrorMessage, IsServerOnline). DI-контейнер регистрирует HTTP-клиенты (ApiService, ApiHealthService), сервисы (AuthService, RouteService, BookingService, PaymentService, TicketService, FavoriteService, QrCodeService, OfflineStorageService) и ViewModel со страницами. AuthService управляет JWT [7]-токеном с хранением в SecureStorage и уведомляет подписчиков через AuthStateChanged.

Модели данных соответствуют контрактам API. SeatItemModel реализует INotifyPropertyChanged для реактивного обновления схемы салона (доступно — зелёный, выбрано — синий, занято — серый).

BookingViewModel реализует четырёхшаговый мастер бронирования: выбор конкретного рейса, указание количества мест с автоматическим пересчётом стоимости, интерактивная схема салона автобуса с цветовой индикацией мест, ввод данных пассажира. Шаги содержат валидацию, пользователь может переключаться между ними как вперёд, так и назад с сохранением введённых данных. После успешного создания бронирования ViewModel переходит в состояние завершения, и пользователю предлагается перейти к оплате. Процесс бронирования в мобильном приложении показан на рисунке 18.

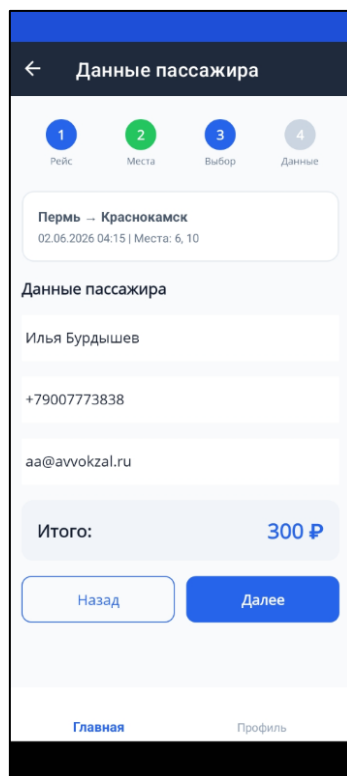


Рисунок 18 — Страница бронирования

PaymentViewModel загружает информацию о бронировании, предоставляет выбор метода оплаты (банковская карта, СБП, наличные) и обрабатывает платеж через API. При успешной оплате статус бронирования изменяется на CONFIRMED, и система генерирует электронные билеты. TicketViewModel загружает созданные билеты и формирует QR-код для каждого с использованием библиотеки QRCoder [16] с уровнем коррекции ошибок Q, после чего сохраняет их в офлайн-кэш через OfflineStorageService. Страница электронного билета показана на рисунке 19.

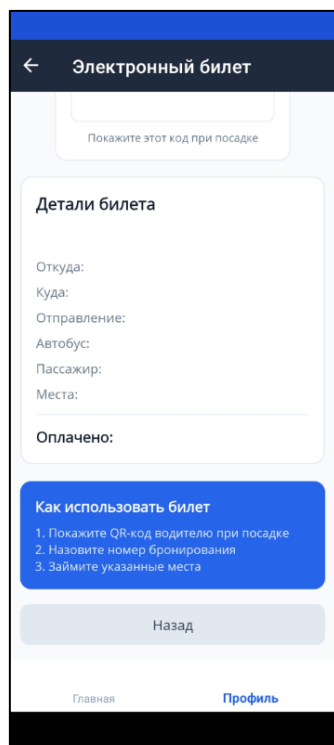


Рисунок 19 — Страница электронного билета

Ключевым преимуществом является поддержка автономного режима. `OfflineStorageService` кэширует до 10 билетов в JSON-файл, при недоступности сервера загрузка выполняется из кэша. `ApiHealthService` проверяет доступность сервера и переключает режимы онлайн/офлайн через событие `ConnectivityChanged`.

Навигация между страницами приложения организована через Shell-навигацию .NET MAUI [9] с `TabBar`, содержащим вкладки «Главная» и «Профиль». Дополнительные маршруты (`login`, `register`, `booking`, `payment`, `ticket`) зарегистрированы через `Routing.RegisterRoute` и доступны для программной навигации с передачей параметров через `QueryPropertyAttribute`. Пользовательский интерфейс каждой страницы описан в XAML с привязкой данных к `ViewModel` и использованием конвертеров (`StatusToColor`, `StatusToText`, `PriceFormat`). Единая цветовая палитра и стили элементов управления централизованно заданы в файлах `Colors.xaml` и `Styles.xaml`. Страница профиля пользователя показана на рисунке 20.

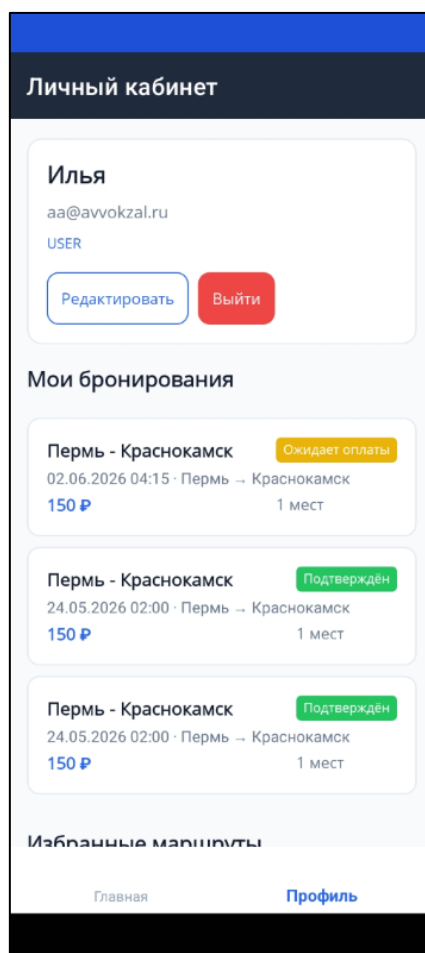


Рисунок 20 — Страница профиля пользователя

Главная страница мобильного приложения предоставляет доступ к поиску маршрутов по пунктам отправления и назначения, отображению списка доступных направлений и мониторингу доступности сервера. При потере соединения пользователь получает предупреждение об ограничении функциональности. Интерфейс главной страницы выполнен в едином стиле с веб-приложением. Главная страница мобильного приложения с поиском маршрутов показана на рисунке 21.

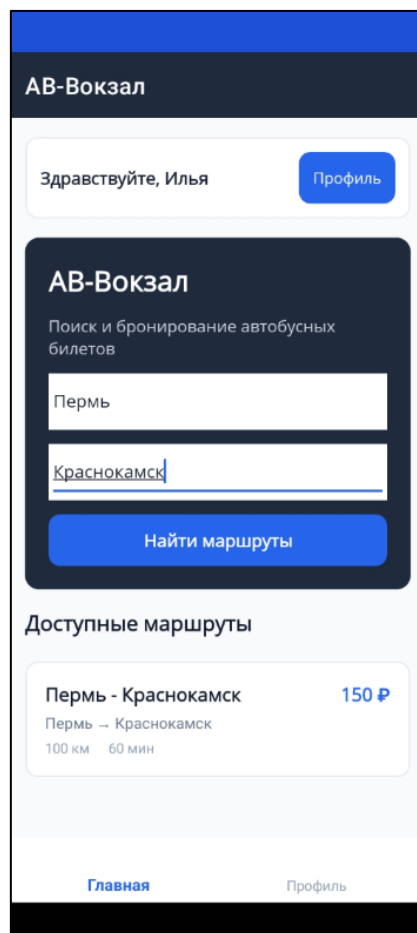


Рисунок 21 — Главная страница с поиском маршрутов

Приложение реализует полный цикл обслуживания: от поиска маршрута до предъявления QR-кода при посадке, включая автономный режим для критически важной функции доступа к билетам без интернета.

Вывод по практической части

В практической части рассмотрены этапы проектирования и разработки информационной системы «AB-Вокзал». Проектирование выполнено на основе трёхзвенной клиент-серверной архитектуры с REST API [18], ролевой моделью доступа на четыре роли и двухуровневой проверкой прав (серверная через JWT-гарды, клиентская через NextJS middleware). Спроектированы процессы аутентификации, бронирования, оплаты, чата и отслеживания задержек.

Разработана база данных PostgreSQL [13] с двенадцатью моделями и нормализацией до третьей нормальной формы. Реализован серверный API на NestJS [10] с модулями аутентификации, маршрутов, рейсов, бронирования,

					BKP 09.02.07.22.4.2.3	Лист
Изм.	Лист	№ документа	Подпись	Дата		46

билетов, платежей, чата, задержек, избранного и пользователей. Создано веб-приложение на NextJS [11] с PrimeReact [14] для всех ролей. Разработано мобильное приложение на .NET MAUI [9] с пошаговым мастером бронирования, онлайн-оплатой, генерацией QR-кодов и поддержкой автономного доступа к электронным билетам.

Практическая реализация подтверждает работоспособность выбранной архитектуры и технологического стека. Система обеспечивает полный цикл обслуживания пассажиров — от поиска маршрута до предъявления электронного билета при посадке — и предоставляет инструменты оперативного управления для персонала автовокзала.

					<i>ВКР 09.02.07.22.4.2.3</i>	Лист
Изм.	Лист	№ документа	Подпись	Дата		47

3 ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ

Экономическое обоснование разработки информационной системы «АВ-Вокзал» направлено на определение затрат, связанных с созданием программного продукта, а также на оценку экономической целесообразности его внедрения. Разработка системы позволяет автоматизировать процессы бронирования и оплаты билетов, управления маршрутами и расписаниями, взаимодействия между пассажирами и персоналом.

Состав и время выполнения работ по созданию информационной системы «АВ-Вокзал»

Разработка системы выполнялась в течение 55 календарных дней при средней занятости разработчика 7 часов в день. Общее количество затраченного времени составило 380 часов. Состав и время выполнения работ представлены в таблице 4.

Таблица 4 — Состав и время выполнения работ

<i>Состав работ</i>	<i>Время выполнения</i>
<i>Установка и настройка программного обеспечения</i>	5 часов
<i>Анализ предметной области и аналогичных решений</i>	20 часов
<i>Проектирование архитектуры системы</i>	25 часов
<i>Разработка дизайна пользовательского интерфейса</i>	30 часов
<i>Разработка серверной части (NestJS API)</i>	70 часов
<i>Проектирование и реализация базы данных (PostgreSQL)</i>	20 часов
<i>Разработка веб-приложения (NextJS)</i>	70 часов
<i>Разработка мобильного приложения (.NET MAUI)</i>	50 часов
<i>Тестирование функционала</i>	20 часов
<i>Отладка и исправление ошибок</i>	25 часов
<i>Оптимизация производительности и безопасности</i>	15 часов
<i>Написание диплома</i>	30 часов
<i>Итого:</i>	380 часов

Расчет себестоимости

Себестоимость — это расходы предприятий, непосредственно связанные с производством, покупкой и реализацией продукции, выполнением работ и предоставлением услуг.

Выявление резервов снижения себестоимости должно опираться на комплексный технико-экономический анализ работы предприятия: изучение технического и организационного уровня производства, использование производственных мощностей и основных фондов, сырья и материалов, рабочей силы, хозяйственных связей.

$$C = Z_{\text{пр}} + R_{\text{накл}} \quad (1)$$

Где C — себестоимость продукта;

$Z_{\text{пр}}$ — прямые затраты, непосредственно связанные с разработкой;

$R_{\text{накл}}$ — накладные расходы, обеспечивающие выполнение работ.

Накладные расходы в расчетах принимаются в размере 30% от суммы прямых затрат, что является общепринятой практикой при экономическом анализе программных проектов:

$$R_{\text{накл}} = 0,3 \cdot Z_{\text{пр}} \quad (2)$$

Прямые затраты включают расходы на материалы, условную заработную плату разработчика и затраты на содержание и эксплуатацию оборудования:

$$Z_{\text{пр}} = M + Z_{\text{п}} + PC_{\text{ЭО}} \quad (3)$$

Расчет затрат по статье расходов «Материалы»

Несмотря на то, что разработка информационной системы в основном связана с интеллектуальным трудом, в процессе выполнения работы также использовались различные материальные ресурсы. К ним относятся расходные материалы, необходимые для подготовки отчетной документации, хранения данных и организации рабочего процесса.

Смета затрат на материалы представлена в таблице 5.

Таблица 5 — Смета затрат на материалы

Наименование материала	Ед. изм.	Кол-во	Цена, руб.	Стоимость, руб.
Бумага формата А4 для печати	пачка	2	400	800
USB-накопитель	шт	1	1 200	1 200
Канцелярские принадлежности	комплект	1	700	700
Итого:				2 700

Таким образом, суммарные затраты на материалы составили:

$$M = 2\,700 \text{ руб.}$$

Расчет фонда заработной платы

Разработка информационной системы «АВ-Вокзал» выполнялась в рамках выпускной квалификационной работы, поэтому фактическая выплата заработной платы разработчику не производилась. Однако для получения объективной экономической оценки выполнен условный расчет фонда заработной платы, отражающий рыночную стоимость аналогичных работ.

Фонд заработной платы рассчитывается по формуле:

$$\text{ФЗП} = \text{Зч} \cdot T \quad (5)$$

где:

ФЗП — фонд заработной платы, руб.;

Зч — средняя часовая ставка разработчика, руб./час;

T — суммарное время выполнения работ, час.

Средняя часовая ставка веб-разработчика, исходя из анализа рынка труда, принимается равной:

$$\text{Зч} = 600 \text{ руб./час}$$

Общее время разработки составляет:

$$T = 380 \text{ часов}$$

Тогда фонд заработной платы составит:

$$\text{ФЗП} = 600 \cdot 380 = 228\,000 \text{ руб.}$$

Дополнительно фонд заработной платы может быть представлен через среднемесячную заработную плату:

$$\text{Зм} = \text{Зч} \cdot t_m \quad (6)$$

где:

Z_m — среднемесячная заработная плата;

t_m — среднемесячное количество рабочих часов.

Принимая $t_m = 168$ часов, получаем:

$$Z_m = 600 \cdot 168 = 100\,800 \text{ руб.}$$

Расчет затрат на содержание и эксплуатацию оборудования

Затраты на содержание и эксплуатацию оборудования представляют собой совокупность расходов, связанных с использованием технических средств и программного обеспечения в процессе разработки информационной системы «АВ-Вокзал». Данные затраты включают амортизацию оборудования, затраты на электроэнергию, износ малоценного инструмента, расходы на обслуживание и ремонт оборудования, а также прочие сопутствующие расходы.

Общая сумма затрат на содержание и эксплуатацию оборудования определяется по формуле:

$$PCЭО = A_{об} + Z_{ээ} + Z_{инстр} + Z_{всп} + Z_{рем} + Z_{проч} \quad (6)$$

Где:

$A_{об}$ — затраты на амортизацию оборудования;

$Z_{ээ}$ — затраты на электроэнергию;

$Z_{инстр}$ — затраты на износ малоценного инструмента;

$Z_{всп}$ — затраты на вспомогательные материалы;

$Z_{рем}$ — затраты на ремонт и обслуживание оборудования;

$Z_{проч}$ — прочие затраты.

Расчет амортизации оборудования

Амортизация оборудования рассчитывается исходя из первоначальной стоимости компьютерной техники и установленной нормы амортизации:

$$A_{об.год} = C_{п} \cdot N_{а} \quad (7)$$

Где:

$C_{п}$ — первоначальная стоимость оборудования;

На — годовая норма амортизации.

Принимаем первоначальную стоимость компьютера $C_{п} = 80\,000$ руб.,
норма амортизации $Н_{а} = 20\%$.

$$A_{об.год} = 80\,000 \cdot 0,2 = 16\,000 \text{ руб.}$$

Так как разработка велась в течение 55 дней, амортизация за период разработки определяется по формуле:

$$A_{об} = A_{об.год} \cdot K_{д} / 365 \quad (8)$$

Где:

$K_{д}$ — количество дней разработки.

$$A_{об} = 16\,000 \cdot 55 / 365 \approx 2\,411 \text{ руб.}$$

Расчет затрат на электроэнергию

Затраты на электроэнергию рассчитываются по формуле:

$$З_{ээ} = T \cdot \mathcal{E} \cdot t_{э} \quad (9)$$

Где:

T — общее время работы за компьютером, часов;

$\mathcal{E}$ — средняя потребляемая мощность оборудования, кВт/ч;

$t_{э}$ — тариф на электроэнергию.

При значениях

$$T = 380 \text{ часов,}$$

$$\mathcal{E} = 0,65 \text{ кВт/ч,}$$

$$t_{э} = 5,5 \text{ руб./кВт}\cdot\text{ч,}$$

получаем:

$$З_{ээ} = 380 \cdot 0,65 \cdot 5,5 \approx 1\,359 \text{ руб.}$$

Расчет затрат на износ малоценного инструмента

Затраты на износ малоценного инструмента принимаются исходя из среднегодового значения затрат на одно рабочее место:

$$З_{инстр} = З_{инстр.год} \cdot K_{д} / 365 \quad (10)$$

Где:

$З_{инстр.год}$ — годовые затраты на износ инструмента.

Принимаем $Z_{\text{инстр.год}} = 4\,000$ руб.

$Z_{\text{инстр}} = 4\,000 \cdot 55 / 365 \approx 603$ руб.

Расчет затрат на вспомогательные материалы

Затраты на вспомогательные материалы определяются аналогично:

$Z_{\text{всп}} = Z_{\text{всп.год}} \cdot K_d / 365$ (11)

Где:

$Z_{\text{всп.год}}$ — годовые затраты на вспомогательные материалы.

Принимаем $Z_{\text{всп.год}} = 3\,500$ руб.

$Z_{\text{всп}} = 3\,500 \cdot 55 / 365 \approx 527$ руб.

Расчет затрат на ремонт и обслуживание оборудования

Затраты на ремонт и обслуживание оборудования определяются как доля от первоначальной стоимости:

$Z_{\text{рем}} = C_p \cdot N_{\text{рем}} \cdot K_d / 365$ (13)

Где:

$N_{\text{рем}}$ — норматив затрат на ремонт и обслуживание оборудования.

При $N_{\text{рем}} = 7\%$:

$Z_{\text{рем}} = 80\,000 \cdot 0,07 \cdot 55 / 365 \approx 844$ руб.

Расчет прочих затрат

Прочие затраты принимаются в размере 8% от суммы основных затрат:

$Z_{\text{проч}} = (A_{\text{об}} + Z_{\text{ээ}} + Z_{\text{инстр}} + Z_{\text{всп}} + Z_{\text{рем}}) \cdot 0,08$ (14)

$Z_{\text{проч}} = (2\,411 + 1\,359 + 603 + 527 + 844) \cdot 0,08 \approx 460$ руб.

Итоговые затраты на содержание и эксплуатацию оборудования

Подставляя полученные значения в формулу (6), получаем:

$PC_{\text{ЭО}} = 2\,411 + 1\,359 + 603 + 527 + 844 + 460 \approx 6\,204$ руб.

Округляя значение, принимаем:

$PC_{\text{ЭО}} \approx 6\,200$ руб.

Расчет прямых и накладных затрат

					<i>БКР 09.02.07.22.4.2.3</i>	<i>Лист</i>
						53
<i>Изм.</i>	<i>Лист</i>	<i>№ документа</i>	<i>Подпись</i>	<i>Дата</i>		

Прямые затраты на разработку информационной системы «АВ-Вокзал» определяются по формуле:

$$З_{пр} = М + З_{п} + РСЭО \quad (15)$$

$$З_{пр} = 2\,700 + 228\,000 + 6\,200 = 236\,900 \text{ руб.}$$

Накладные расходы рассчитываются как 30% от прямых затрат:

$$Р_{накл} = 0,3 \cdot З_{пр} \quad (16)$$

$$Р_{накл} = 0,3 \cdot 236\,900 = 71\,070 \text{ руб.}$$

Себестоимость разработки информационной системы «АВ-Вокзал»

Себестоимость разработки определяется по формуле:

$$С = З_{пр} + Р_{накл} \quad (17)$$

$$С = 236\,900 + 71\,070 = 307\,970 \text{ руб.}$$

Экономическая эффективность проекта

Экономическая эффективность разработки информационной системы «АВ-Вокзал» определяется как разница между стоимостью разработки у стороннего подрядчика и себестоимостью собственного проекта:

$$\Delta = Ц_{созд} - С \quad (18)$$

Где:

Ц_{созд} — стоимость разработки системы профессиональной веб-студией;

С — себестоимость разработки.

При Ц_{созд} = 600 000 руб. получаем:

$$\Delta = 600\,000 - 307\,970 = 292\,030 \text{ руб.}$$

По результатам выполненных расчетов экономической эффективности проектирования и внедрения информационной системы «АВ-Вокзал» можно сделать вывод о целесообразности и экономической оправданности данного решения. Несмотря на то, что полученная экономическая выгода носит в основном косвенный характер, в среднесрочной и долгосрочной перспективе

эффект от внедрения автоматизированного решения становится заметным и устойчивым.

Разработка и внедрение информационной системы «АВ-Вокзал» способствует автоматизации ключевых бизнес-процессов автовокзала, таких как бронирование и оплата билетов, управление маршрутами и расписаниями, отслеживание задержек рейсов, коммуникация между водителями и операторами через встроенный чат, а также формирование электронных билетов с QR-кодами для посадки. За счет этого значительно сокращается время на обслуживание пассажиров, снижается нагрузка на персонал и уменьшается вероятность ошибок, связанных с ручной обработкой данных.

Внедрение средств автоматизации позволяет оптимизировать работу автовокзала, что в перспективе может привести либо к сокращению затрат на персонал, либо к увеличению объема обслуживаемых пассажиров при сохранении прежней численности сотрудников. Это особенно важно в условиях конкуренции между транспортными предприятиями, когда повышение эффективности работы напрямую влияет на финансовые показатели и качество обслуживания.

Практика показывает, что автоматизация процессов, в том числе взаимодействия с пассажирами через веб-сайт и мобильное приложение, управления расписанием и мониторинга рейсов, обладает значительным потенциалом для развития и получения материальной выгоды с течением времени. Использование информационной системы «АВ-Вокзал» как единой платформы позволяет централизовать данные о маршрутах, рейсах и пассажирах, упростить навигацию по сервисам и обеспечить быстрый доступ к актуальной информации для всех участников перевозочного процесса — пассажиров, администраторов, операторов и водителей.

При расчете экономической эффективности необходимо учитывать важное свойство автоматизации: чем более тщательно и качественно проработан программный продукт на этапе проектирования и внедрения, тем выше экономический эффект в процессе его дальнейшей эксплуатации.

Детальная проработка архитектуры системы, структуры базы данных и логики работы серверной части, веб-приложения и мобильного приложения снижает затраты на доработку, сопровождение и техническую поддержку в будущем.

Кроме того, использование единой информационной системы для автоматизации работы автовокзала позволяет сократить временные и материальные затраты, связанные с организацией хранения и обработки данных о маршрутах, рейсах, бронированиях и пассажирах. Снижаются издержки на поиск и бронирование билетов, повышается качество обслуживания и удовлетворённость пассажиров.

Таким образом, внедрение разработанной информационной системы «АВ-Вокзал» является экономически обоснованным и эффективным решением, способствующим повышению качества обслуживания пассажиров и улучшению эффективности работы персонала автовокзала.

4 ПРАВИЛА И НОРМЫ ТЕХНИКИ БЕЗОПАСНОСТИ

Общие требования техники безопасности

Разработка и сопровождение информационной системы «АВ-Вокзал» осуществляется с использованием персональных компьютеров, сетевого оборудования и программных средств, что связано с воздействием потенциально вредных и опасных факторов на пользователя. К таким факторам относятся электрическое напряжение, электромагнитное излучение, повышенная нагрузка на органы зрения и опорно-двигательный аппарат, а также психоэмоциональное напряжение.

Несоблюдение установленных правил и норм техники безопасности при работе с вычислительной техникой может привести к поражению электрическим током, возникновению пожароопасных ситуаций, повреждению оборудования, а также негативно сказаться на здоровье разработчика. В связи с этим при разработке информационной системы «АВ-Вокзал» необходимо строго соблюдать действующие санитарно-гигиенические нормы и требования охраны труда.

Требования к организации рабочего места разработчика

Рабочее место разработчика информационной системы «АВ-Вокзал» должно быть организовано таким образом, чтобы обеспечивать безопасные и комфортные условия труда. Рабочее место, оборудованное персональным компьютером, должно предусматривать:

- 1) наличие исправного заземления всех компонентов вычислительной техники, подключаемых к электрической сети (системный блок, монитор, периферийные устройства);
- 2) использование сетевого фильтра или источника бесперебойного питания для защиты оборудования от перепадов напряжения;
- 3) применение современных жидкокристаллических мониторов, соответствующих требованиям санитарных норм по уровню электромагнитного излучения;

					<i>ВКР 09.02.07.22.4.2.3</i>	Лист
						57
Изм.	Лист	№ документа	Подпись	Дата		

4) наличие достаточного уровня естественного и искусственного освещения без образования бликов на экране монитора;

5) размещение кабелей питания и передачи данных таким образом, чтобы исключить возможность механических повреждений и случайного зацепления.

Помещение, в котором осуществляется разработка и администрирование информационной системы, должно быть оборудовано средствами пожаротушения и аптечкой первой медицинской помощи.

Требования безопасности перед началом работы

Перед началом работы по разработке или администрированию информационной системы «АВ-Вокзал» пользователь обязан:

1) осмотреть рабочее место и убедиться в отсутствии посторонних предметов на столе;

2) проверить исправность компьютерного оборудования и корректность его подключения к электрической сети;

3) убедиться в отсутствии повреждений изоляции кабелей и наличии заземления;

4) отрегулировать положение монитора, клавиатуры и мыши в соответствии с эргономическими требованиями;

5) настроить уровень освещенности рабочего места;

6) убедиться в исправности программного обеспечения и отсутствии ошибок загрузки операционной системы.

Запрещается приступать к работе при наличии неисправностей оборудования или отсутствии защитных средств электробезопасности.

Требования безопасности во время работы

Во время работы с персональным компьютером при разработке информационной системы «АВ-Вокзал» пользователь обязан:

- 1) выполнять только те операции, которые соответствуют поставленным задачам разработки и администрирования сайта;
- 2) соблюдать чистоту и порядок на рабочем месте;
- 3) не закрывать вентиляционные отверстия системного блока и монитора;
- 4) не допускать попадания влаги на элементы вычислительной техники;
- 5) соблюдать режим труда и отдыха, делая регламентированные перерывы для снижения зрительной и физической нагрузки;
- 6) соблюдать расстояние от глаз до экрана монитора в пределах 60–70 см;
- 7) использовать эргономичную посадку и избегать длительного статического напряжения мышц.

Пользователю запрещается:

- 1) производить подключение или отключение периферийных устройств при включенном питании;
- 2) прикасаться к внутренним компонентам компьютера;
- 3) самостоятельно вскрывать и ремонтировать оборудование;
- 4) использовать неисправное или поврежденное оборудование.

Требования безопасности в аварийных ситуациях

В случае возникновения аварийных ситуаций при работе с вычислительной техникой пользователь обязан:

- 1) немедленно отключить питание оборудования при появлении запаха гари, дыма или искрения;
- 2) сообщить о неисправности ответственному лицу или администратору;
- 3) прекратить работу при возникновении ухудшения самочувствия (резь в глазах, головокружение, боли в руках);

- 4) не возобновлять работу до полного устранения выявленных неисправностей;
- 5) при необходимости оказать первую медицинскую помощь и вызвать экстренные службы.

Требования безопасности после окончания работы

После завершения работы по разработке или обслуживанию информационной системы «АВ-Вокзал» пользователь обязан:

- 1) корректно завершить работу всех программных приложений;
- 2) сохранить результаты выполненной работы;
- 3) выключить питание системного блока и периферийных устройств;
- 4) отключить оборудование от электросети;
- 5) привести рабочее место в порядок и выполнить упражнения для снятия зрительного и мышечного напряжения.

Соблюдение правил техники безопасности при разработке информационной системы «АВ-Вокзал» — необходимое условие для создания безопасных условий труда. Оно минимизирует вредные факторы (напряжение, излучение, нагрузку на зрение и опорно-двигательный аппарат) и способствует сохранению здоровья разработчика.

Правильная организация рабочего места и эксплуатация оборудования снижают утомляемость, поддерживают работоспособность и повышают производительность, что напрямую влияет на качество и сроки создания информационной системы.

Кроме того, соблюдение норм предотвращает аварии, потерю данных и пожароопасные ситуации, а также формирует ответственное отношение к безопасности при работе с ИТ-системами.

В итоге, это обеспечивает не только личную безопасность и сохранность оборудования, но и стабильную работу всей информационной системы.

ЗАКЛЮЧЕНИЕ

В рамках данной работы разработана информационная система «АВ-Вокзал», автоматизирующая процессы бронирования и оплаты билетов, управления маршрутами и расписаниями, а также взаимодействия между пассажирами, водителями и операторами автовокзала.

В теоретической части рассмотрена механика работы информационной системы, проведён анализ существующих аналогов, обоснован выбор технологического стека и платформы развёртывания. Анализ показал отсутствие на рынке решений, сочетающих веб-интерфейс и мобильное приложение с поддержкой офлайн-режима и QR-кодами для пассажирских перевозок, что подтверждает актуальность разработки.

В практической части спроектирована и реализована трёхзвенная клиент-серверная архитектура системы. Разработана реляционная база данных на PostgreSQL с двенадцатью моделями и нормализацией до третьей нормальной формы. Реализован серверный API на базе NestJS с модульной архитектурой, включающий модули аутентификации, маршрутов, рейсов, бронирования, билетов, платежей, чата, задержек, избранного и пользователей. Создано веб-приложение на NextJS с PrimeReact для администраторов, операторов и водителей. Разработано мобильное приложение на .NET MAUI с паттерном MVVM для пассажиров, поддерживающее бронирование билетов, онлайн-оплату, отображение QR-кодов и офлайн-доступ к электронным билетам.

Таким образом, задачи, поставленные в начале работы, выполнены в полном объёме. Разработанная информационная система «АВ-Вокзал» обеспечивает полный цикл обслуживания пассажиров — от поиска маршрута до предъявления электронного билета при посадке — и предоставляет инструменты эффективного управления для персонала автовокзала.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- 1) Axios — документация HTTP-клиента [Электронный ресурс]. — URL: <https://axios-http.com/docs/intro> (дата обращения: 20.03.2026);
- 2) bcrypt — алгоритм хеширования паролей [Электронный ресурс]. — URL: <https://github.com/kelektiv/node.bcrypt.js> (дата обращения: 20.03.2026);
- 3) class-validator — документация библиотеки валидации для TypeScript [Электронный ресурс]. — URL: <https://github.com/typestack/class-validator> (дата обращения: 20.03.2026);
- 4) CommunityToolkit.MVVM — документация библиотеки [Электронный ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/> (дата обращения: 20.03.2026);
- 5) Docker — документация платформы контейнеризации [Электронный ресурс]. — URL: <https://docs.docker.com/> (дата обращения: 20.03.2026);
- 6) Docker Compose — документация инструмента оркестрации контейнеров [Электронный ресурс]. — URL: <https://docs.docker.com/compose/> (дата обращения: 20.03.2026);
- 7) JSON Web Token (JWT) — стандарт RFC 7519 [Электронный ресурс]. — URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 20.03.2026);
- 8) MVVM — паттерн проектирования Model-View-ViewModel [Электронный ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/architecture/maui/mvvm> (дата обращения: 20.03.2026);
- 9) NET MAUI — документация кроссплатформенного фреймворка [Электронный ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/maui/> (дата обращения: 20.03.2026);
- 10) NestJS — документация серверного фреймворка [Электронный ресурс]. — URL: <https://docs.nestjs.com/> (дата обращения: 20.03.2026);
- 11) NextJS — документация фреймворка [Электронный ресурс]. — URL: <https://nextjs.org/docs> (дата обращения: 20.03.2026);

12) Passport.js — документация middleware аутентификации для Node.js [Электронный ресурс]. — URL: <https://www.passportjs.org/> (дата обращения: 20.03.2026);

13) PostgreSQL — официальная документация СУБД [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 20.03.2026);

14) PrimeReact — документация библиотеки UI-компонентов [Электронный ресурс]. — URL: <https://primereact.org/> (дата обращения: 20.03.2026);

15) Prisma — документация ORM для Node.js и TypeScript [Электронный ресурс]. — URL: <https://www.prisma.io/docs> (дата обращения: 20.03.2026);

16) QRCode — библиотека генерации QR-кодов для .NET [Электронный ресурс]. — URL: <https://github.com/codebude/QRCode> (дата обращения: 20.03.2026);

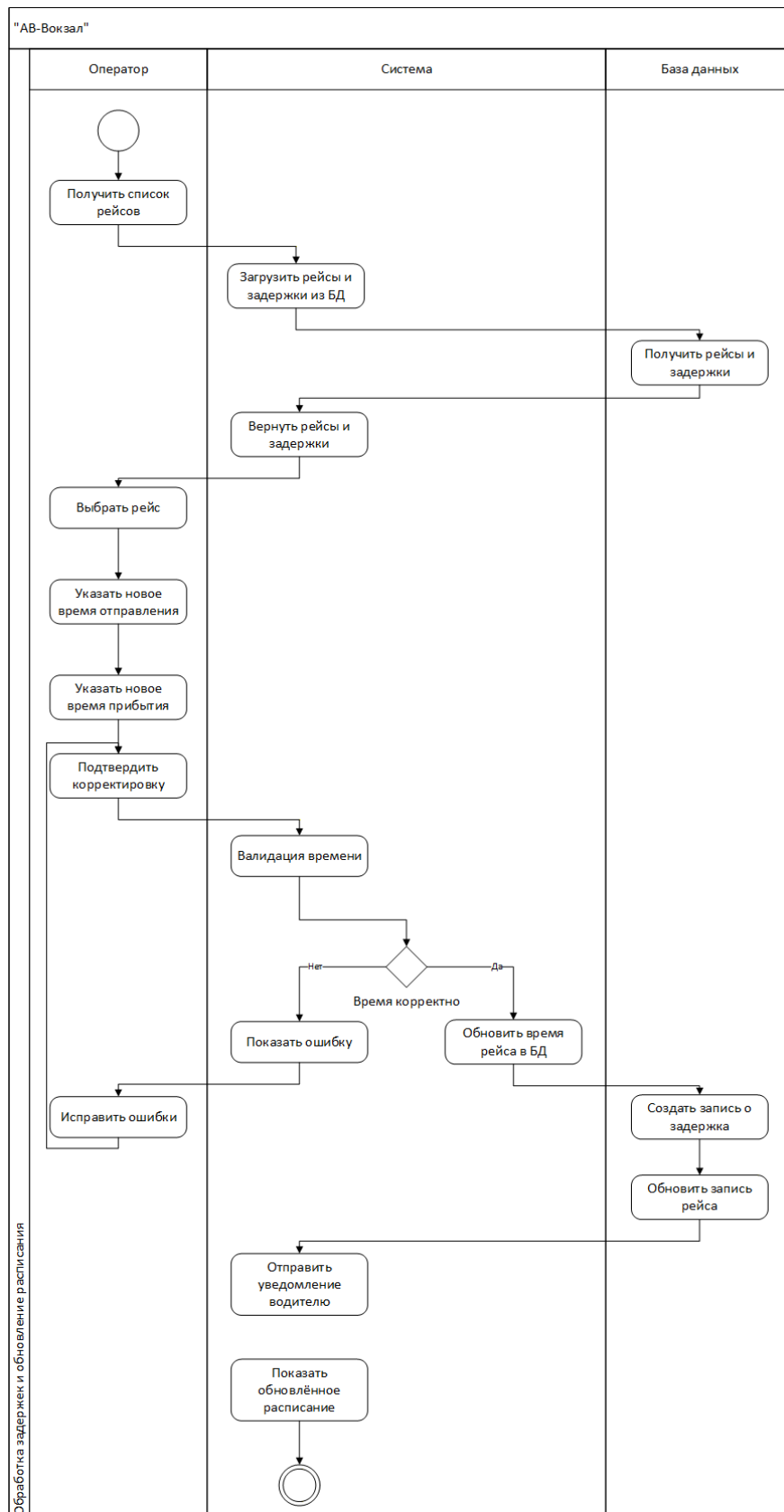
17) React — документация библиотеки [Электронный ресурс]. — URL: <https://react.dev/learn> (дата обращения: 20.03.2026);

18) REST — архитектурный стиль проектирования API [Электронный ресурс]. — URL: https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (дата обращения: 20.03.2026);

19) TailwindCSS — документация утилитарного CSS-фреймворка [Электронный ресурс]. — URL: <https://tailwindcss.com/docs> (дата обращения: 20.03.2026);

20) TypeScript — документация языка программирования [Электронный ресурс]. — URL: <https://www.typescriptlang.org/docs/> (дата обращения: 20.03.2026);

Диаграмма деятельности механизма обработки задержек и обновления расписания



ПРИЛОЖЕНИЕ Б

Спецификация основных конечных точек API

Метод	Конечная точка	Описание	Защита
POST	/api/auth/register	Регистрация нового пользователя	Публичный
POST	/api/auth/login	Аутентификация и получение JWT-токена	Публичный
GET	/api/auth/validate	Проверка валидности JWT-токена	Публичный
GET	/api/auth/profile	Получение профиля текущего пользователя	JwtAuthGuard
GET	/api/routes	Получение списка всех маршрутов	Публичный
GET	/api/routes/search	Поиск маршрутов по пунктам отправления и назначения	Публичный
GET	/api/routes/:id	Получение маршрута по идентификатору	Публичный
POST	/api/routes	Создание нового маршрута	JwtAuthGuard
PUT	/api/routes/:id	Обновление маршрута	JwtAuthGuard
DELETE	/api/routes/:id	Удаление маршрута	JwtAuthGuard
GET	/api/trips	Получение списка всех рейсов	Публичный
GET	/api/trips/route/:routeId	Получение рейсов по маршруту	Публичный
GET	/api/trips/:id/seats	Получение количества доступных мест	Публичный
POST	/api/trips	Создание нового рейса	JwtAuthGuard
PUT	/api/trips/:id/time	Обновление времени рейса (оператор)	JwtAuthGuard
GET	/api/bookings	Получение списка бронирований	Публичный
POST	/api/bookings	Создание нового бронирования	JwtAuthGuard
PUT	/api/bookings/:id/confirm	Подтверждение бронирования	JwtAuthGuard
PUT	/api/bookings/:id/cancel	Отмена бронирования	JwtAuthGuard
POST	/api/tickets	Создание билета с генерацией QR-кода	Публичный
GET	/api/tickets/:id	Получение билета по идентификатору	Публичный
POST	/api/tickets/:id/validate	Проверка подлинности билета	Публичный
GET	/api/payments	Получение списка платежей	Публичный
POST	/api/payments	Создание платежа	JwtAuthGuard
POST	/api/payments/:id/process	Обработка платежа	JwtAuthGuard
GET	/api/chat/driver/:driverId	Получение сообщений чата водителя	JwtAuthGuard
POST	/api/chat	Отправка сообщения в чат	JwtAuthGuard
GET	/api/delays	Получение списка задержек	Публичный
POST	/api/delays	Создание записи о задержке	JwtAuthGuard

ПРИЛОЖЕНИЕ В

Сводная информация о технологическом стеке проекта

Технология	Назначение	Уровень	Описание
<i>NextJS</i>	Фреймворк веб-приложений	Фронтенд	React-фреймворк с поддержкой SSR, SSG и API Routes для серверного рендеринга
<i>React</i>	Библиотека пользовательских интерфейсов	Фронтенд	Декларативная библиотека для построения компонентного UI
<i>TailwindCSS</i>	CSS-фреймворк	Фронтенд	Утилитарный CSS-фреймворк для быстрой стилизации компонентов
<i>PrimeReact</i>	Библиотека UI-компонентов	Фронтенд	Набор готовых React-компонентов: таблицы, диалоги, формы, графики
<i>NestJS</i>	Серверный фреймворк	Бэкенд	Модульный Node.js-фреймворк с поддержкой TypeScript, DI и модульной архитектуры
<i>Prisma</i>	ORM-решение	Бэкенд	Schema-first ORM с автогенерацией типизированного клиента и встроенными миграциями
<i>PostgreSQL</i>	Система управления базами данных	Бэкенд	Реляционная СУБД с поддержкой ACID-транзакций, расширенных типов и индексирования
<i>TypeScript</i>	Язык программирования	Фронтенд/Бэкенд	Типизированное надмножество JavaScript для повышения надёжности кода
<i>.NET MAUI</i>	Кроссплатформенный фреймворк	Мобильное приложение	Фреймворк для создания мобильных приложений на C# с поддержкой Android и iOS
<i>JWT</i>	Механизм аутентификации	Бэкенд	JSON Web Tokens для Stateless-аутентификации и авторизации пользователей
<i>Vercel</i>	Платформа развёртывания	Инфраструктура	Облачная платформа для хостинга NextJS-приложений и serverless-функций
<i>Docker</i>	Система контейнеризации	Инфраструктура	Контейнеризация сервисов для обеспечения воспроизводимости окружения

Изм.	Лист	№ документа	Подпись	Дата

ВКР 09.02.07.22.4.2.3

Лист

66

Структура модулей API

<i>Модуль</i>	<i>Контроллер</i>	<i>Сервис</i>	<i>Основная функциональность</i>
<i>AuthModule</i>	AuthController	AuthService	Регистрация, аутентификация, валидация JWT-токена, получение профиля
<i>UsersModule</i>	UsersController	UsersService	CRUD-операции с пользователями, фильтрация по ролям, мягкое удаление
<i>RoutesModule</i>	RoutesController	RoutesService	Управление маршрутами, поиск по пунктам отправления и назначения
<i>TripsModule</i>	TripsController	TripsService	Управление рейсами, фильтрация по маршруту и водителю, расчёт доступных мест
<i>BookingsModule</i>	BookingsController	BookingsService	Создание и управление бронированиями, проверка доступности мест, подтверждение и отмена
<i>TicketsModule</i>	TicketsController	TicketsService	Генерация электронных билетов с QR-кодами, проверка подлинности, отметка об использовании
<i>PaymentsModule</i>	PaymentsController	PaymentsService	Обработка платежей, привязка к бронированиям, возврат средств
<i>FavoritesModule</i>	FavoritesController	FavoritesService	Добавление и удаление маршрутов из избранного, проверка избранного
<i>ChatModule</i>	ChatController	ChatService	Обмен сообщениями между водителями и операторами, отметка о прочтении
<i>DelaysModule</i>	DelaysController	DelaysService	Создание записей о задержках, привязка к рейсам, обновление статуса рейса
<i>PrismaModule</i>	—	PrismaService	Подключение к базе данных PostgreSQL, транзакции, выполнение SQL-запросов
<i>ConfigModule</i>	—	—	Загрузка переменных окружения, конфигурация приложения

[Электронный ресурс]

«АВ-Вокзал»

Сайт: <https://avvokzal-repo.vercel.app/>

Мобильное приложение: <https://github.com/AvengerAnubis/avvokzal-repo/releases/download/release-v1/com.companyname.avtovokzalmobileapp-Signed.apk>

					ВКР 09.02.07.22.4.2.3	Лист
						68
Изм.	Лист	№ документа	Подпись	Дата		